

Course End Project 1

Simulated Web Application Penetration Testing

Nidhi Sharma

Problem Statement

Real-time scenario:

A FinTech company plans to launch a web application for managing personal finances. Before the launch, ethical hackers test a simulated version of the app to identify vulnerabilities that could expose sensitive data. Hidden flags represent critical information, ensuring no real data is at risk during testing. This provides a safe environment to assess potential security flaws.

Using tools like Burp Suite, testers uncover issues like exposed debug directories, SQL injection vulnerabilities, and misconfigured login forms. These weaknesses highlight significant risks that could allow unauthorized access to sensitive information.

The hackers document their findings and recommend fixes to the development team. These include parameter sanitization, input validation, and improved access controls. After implementing these changes, the app undergoes further testing to ensure its security.

Industry Relevance

The following tools used in this project serve specific purposes within the industry:

- Kali Linux: Widely used for penetration testing and ethical hacking due to its comprehensive toolset
- Burp Suite: Industry standard for web application security testing, enabling vulnerability assessment and exploitation
- VirtualBox: Essential for creating isolated environments to safely test and simulate vulnerabilities
- dirb/dirbuster: Popular tools for directory brute-forcing, helping identify hidden directories and files in web applications

- Nmap: A critical tool for network scanning and reconnaissance, widely used in vulnerability assessment
- OWASP Zap: Frequently employed for automated web application security testing to identify vulnerabilities efficiently

Table of Contents

Problem Statement.....	2
Real-time scenario:.....	2
Industry Relevance.....	2
Table of Contents.....	4
Project Overview.....	8
Goal.....	8
Scenario.....	8
Tools Used.....	8
Task Breakdown.....	9
Import the vulnerable VM.....	9
Configure network settings.....	9
Reconnaissance.....	9
Vulnerability Assessment.....	9
Report flags and findings.....	9
Hosting vulnerable site on CentOS VM.....	10
Configure Host-Only Network for Virtual Machines (kali and Vulnerable App).....	11
Create a Host-Only Network in VirtualBox.....	11
Assign VMs to Host-Only Network.....	11
Start both VMs.....	13
Verify Connectivity.....	13
Check IP Addresses.....	13
Test Connectivity.....	13
Reconnaissance & Scanning with Nmap.....	15
Open a terminal in Kali Linux.....	15
Analysing NMap Results.....	16
Open Ports and Services.....	16
Service Version Enumeration.....	16
Recommended Attack Paths.....	17
Planning Next Steps.....	17
Directory Brute-Forcing with Dirb or Dirbuster.....	19
Using Dirb.....	19
Why Directory Brute-Forcing Matters After Nmap.....	21
Example Workflow.....	21

Analysing the result of Dirb.....	21
Planning Our Next steps.....	23
What this result means.....	23
Next steps.....	23
Linking with nmap results.....	24
Manual Exploration of Exposed Targets.....	25
Next Steps.....	28
Analysing /pages/BlogsComponent.html.....	31
Analysing /pages/BlogPostComponent.html.....	31
Analysing /index.html.....	32
General Next Steps.....	32
For further testing:.....	33
Testing with dirb big.txt.....	34
Analysing above output.....	36
Brute Force Attack on admin login page.....	38
Confirm Form Details.....	38
Analysing the Network response after entering the login credentials.	39
Key Points from the Developer Tools.....	39
What This Means for Hydra.....	39
Actions to Take.....	39
If /4dm1n/ Accepts POST:.....	40
Run and Monitor.....	40
Summary:.....	40
If Login Not Actually Implemented:.....	40
Check Open Port FTP.....	41
Step 1: Connect to FTP Server.....	41
Step 2: Login Anonymously.....	41
Step 3: List Files and Directories.....	41
Step 4: Navigate and Download Files.....	42
Step 5: Upload File (Test Write Permission).....	43
Step 6: Summary of Vulnerabilities.....	43
Step 7: Exit FTP.....	44
Burpe Suite.....	45
Burp Suite Introduction.....	45
Why Burp Suite?.....	45
Where is it Used?.....	45

Step-by-Step Guide to Set Up Burp Suite for Penetration Testing.....	46
Launch Burp Suite on Kali Linux.....	46
Configure Browser to Use Burp as Proxy.....	46
For Firefox (default browser in Kali):.....	46
In HTTP Proxy, enter:.....	47
Verify Proxy Configuration.....	48
Notes for HTTPS Sites.....	50
Intercept Traffic.....	50
Manual Crawling.....	53
Browse the Application Like a User.....	53
Admin Page.....	53
Monitor the Site Map in Burp.....	55
Use Burp's "Proxy history" for Deeper Review.....	56
Notes.....	57
4adm1n.....	57
How to test?.....	58
/pages/*.html.....	59
Analysis of /pages/*.html Content.....	61
Hidden Information Assessment.....	61
/css/, /js/.....	61
Analysis of /css/ and /js/app.js.....	63
Burp suite: Vulnerabilities and Recommendations.....	64
OWASP ZAP.....	65
OWASP ZAP Introduction & Setup.....	65
What is OWASP ZAP?.....	65
Why use ZAP after Burp?.....	65
ZAP Attack.....	65
Launch OWASP ZAP.....	65
ZAP Starts:.....	66
ZAP Automated Scan.....	66
ZAP Spider Crawl.....	69
ZAP Active Scan.....	71
New Findings ZAP Discovered (Not Found in Burp):.....	76
Form with Password Field Flagged as "Medium" Risk.....	76
Script/Comment Tags Flagged.....	76
More Comprehensive Site Coverage.....	76
Error Page Analysis.....	76

Authentication Context Analysis.....	77
What This Reveals:.....	77
ZAP's Automated Advantages:.....	77
Burp's Manual Advantages (from earlier):.....	77
Flag 5.....	78
XSS Injection Testing on /4dm1n Admin Login.....	81
Testing Approach.....	81
Observation & Results.....	81
Conclusion.....	81
Evidence Screenshots.....	82
SQL Injection Testing on /4dm1n Page.....	83
Testing Approach.....	83
Observations.....	83
Conclusion.....	84
Evidence Screenshots.....	84
Next Steps in Security Assessment.....	86
Verification and Validation.....	86
Knowledge Sharing and Improvement.....	86
Ongoing Monitoring and Hardening.....	86
OWASP Vulnerabilities and Recommendations.....	87
References.....	89
References for Documentation and Downloads.....	89
References for Security Vulnerability and Recommendations.....	89

Project Overview

Goal

Gain hands-on experience by identifying vulnerabilities and capturing hidden flags in a safe, simulated environment.

Scenario

Test a simulated personal finance web app for security flaws before its public launch—no real sensitive data involved.

Tools Used

Kali Linux, Burp Suite, VirtualBox, dirb/dirbuster, Nmap, and OWASP Zap.

Task Breakdown

Import the vulnerable VM

Download and import the vulnerable web application (a Capture-The-Flag box) into VirtualBox.

Configure network settings

Set up the VM's network and record its IP address.

Reconnaissance

Use tools (like Nmap) to map services and gather system information.

Vulnerability Assessment

Analyze for weaknesses using both manual testing and open-source tools (such as Burp Suite and directory brute-forcing tools).

Report flags and findings

Document all discovered flags and provide recommendations for improving security

Hosting vulnerable site on CentOS VM

We have VirtualBox and CentOS installed, so we will be moving to directly hosting the vulnerable website in CentOS VM.

Running the [Vulnerable App](#) Directly as a VM in VirtualBox (Imported OVA)

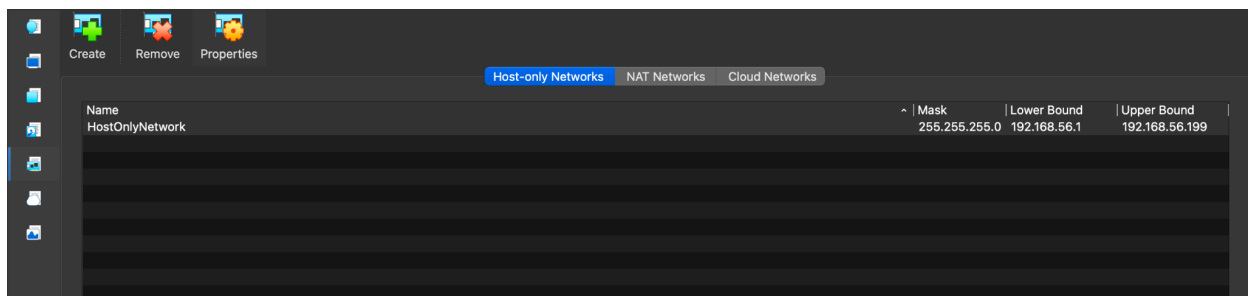
Steps:

- Download the OVA file of the vulnerable app VM.
- Open VirtualBox and use "Import Appliance" to load the OVA.
- Configure the VM as needed (RAM, CPU, network).
- Start the VM to boot into the vulnerable app environment.

Configure Host-Only Network for Virtual Machines (kali and Vulnerable App)

Create a Host-Only Network in VirtualBox

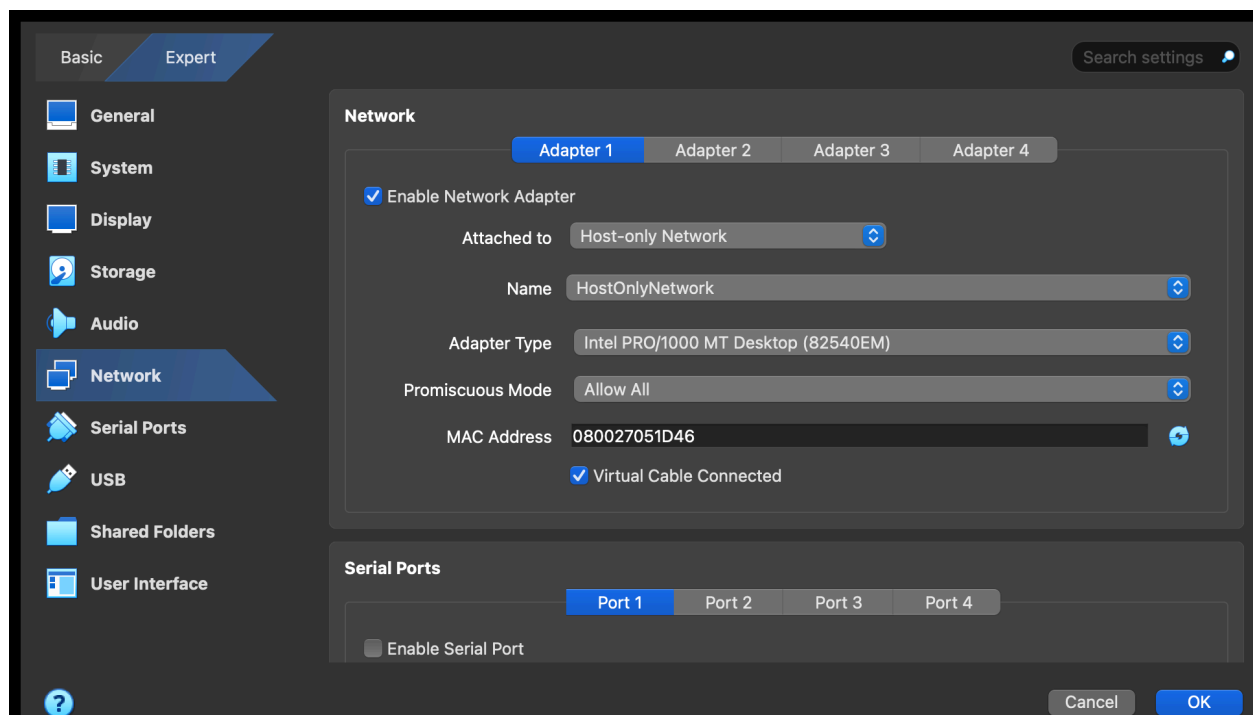
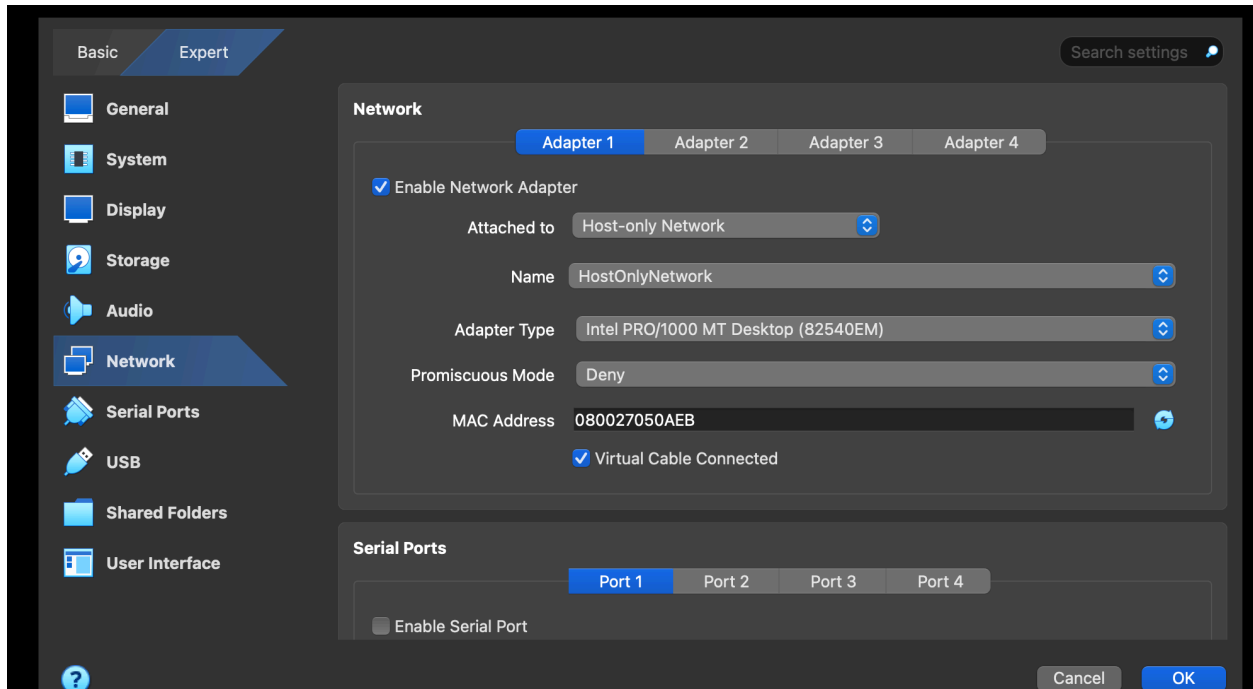
- Open VirtualBox Manager.
- Go to the Left Menu Bar > Network → Oracle VirtualBox Manager
- Check if a host-only network already exists (e.g., vboxnet0). If not, create a new one by clicking on "Create".
- Configure its DHCP settings if needed (usually default works fine).



Assign VMs to Host-Only Network

- For both Kali and the vulnerable VM:
 - Open Settings > Network.
 - Attach Adapter 1 to Host-Only Network.
 - From the dropdown, select the created host-only network (e.g., vboxnet0).

Kali Linux screenshot below and Vulnerable VM Screenshots



Start both VMs

They will get IP addresses from the host-only network and can communicate with each other.

Verify Connectivity

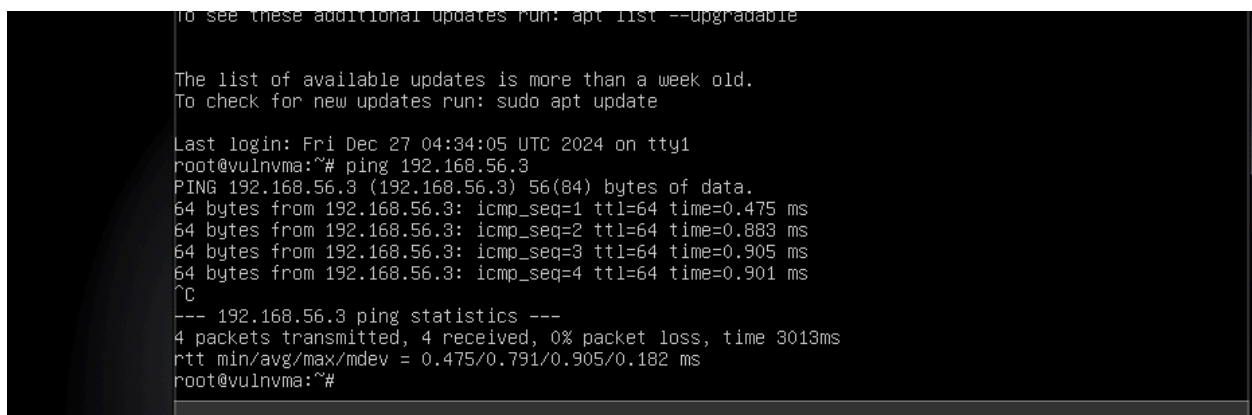
- Check IP on both VMs.
- Ping each other to confirm network access.

Check IP Addresses

- In each VM, run `ifconfig` or `ip a` to get their IPs on the host-only network (usually something like 192.168.56.X).

Test Connectivity

- From Kali, ping the vulnerable VM's IP.
- We should be able to reach the target VM. Now we can proceed with security scans and testing.
-
- Now we can see both can ping each other.



```
to see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Fri Dec 27 04:34:05 UTC 2024 on tty1
root@vulnvma:~# ping 192.168.56.3
PING 192.168.56.3 (192.168.56.3) 56(84) bytes of data:
64 bytes from 192.168.56.3: icmp_seq=1 ttl=64 time=0.475 ms
64 bytes from 192.168.56.3: icmp_seq=2 ttl=64 time=0.883 ms
64 bytes from 192.168.56.3: icmp_seq=3 ttl=64 time=0.905 ms
64 bytes from 192.168.56.3: icmp_seq=4 ttl=64 time=0.901 ms
^C
--- 192.168.56.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3013ms
rtt min/avg/max/mdev = 0.475/0.791/0.905/0.182 ms
root@vulnvma:~#
```

```
valid_ttl forever preferred_ttl forever
(kalimainusersn@kali)-[~]
$ ping 192.168.56.2
PING 192.168.56.2 (192.168.56.2) 56(84) bytes of data.
64 bytes from 192.168.56.2: icmp_seq=1 ttl=64 time=1.48 ms
64 bytes from 192.168.56.2: icmp_seq=2 ttl=64 time=1.49 ms
64 bytes from 192.168.56.2: icmp_seq=3 ttl=64 time=1.23 ms
64 bytes from 192.168.56.2: icmp_seq=4 ttl=64 time=1.27 ms
64 bytes from 192.168.56.2: icmp_seq=5 ttl=64 time=1.16 ms
64 bytes from 192.168.56.2: icmp_seq=6 ttl=64 time=0.883 ms
64 bytes from 192.168.56.2: icmp_seq=7 ttl=64 time=0.963 ms
64 bytes from 192.168.56.2: icmp_seq=8 ttl=64 time=0.777 ms
64 bytes from 192.168.56.2: icmp_seq=9 ttl=64 time=0.909 ms
64 bytes from 192.168.56.2: icmp_seq=10 ttl=64 time=1.08 ms
64 bytes from 192.168.56.2: icmp_seq=11 ttl=64 time=0.729 ms
64 bytes from 192.168.56.2: icmp_seq=12 ttl=64 time=0.944 ms
64 bytes from 192.168.56.2: icmp_seq=13 ttl=64 time=0.933 ms
64 bytes from 192.168.56.2: icmp_seq=14 ttl=64 time=1.06 ms
64 bytes from 192.168.56.2: icmp_seq=15 ttl=64 time=1.56 ms
64 bytes from 192.168.56.2: icmp_seq=16 ttl=64 time=1.52 ms
```



Reconnaissance & Scanning with Nmap

Purpose: Discover Open ports and Services running on Vulnerable VM

Open a terminal in Kali Linux.

Run the command:

```
nmap -sS -sV -A 192.168.56.2
```

`-sS` for stealth SYN scan

`-sV` to detect service versions

`-A` to enable OS and version detection, script scanning, and traceroute

Analyze open ports (e.g., 80 for HTTP, 443 for HTTPS) and running services.

```
(kalimainuserns@kali)-[~]
$ nmap -sS -sV -A 192.168.56.2
Starting Nmap 7.95 ( https://nmap.org ) at 2025-09-19 11:14 EDT
Nmap scan report for 192.168.56.2
Host is up (0.0010s latency).
Not shown: 994 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
20/tcp    closed ftp-data
21/tcp    open  ftp          vsftpd 3.0.5
| ftp-syst:
|_ STAT:
|_ FTP server status:
|_   Connected to ::ffff:192.168.56.3
|_   Logged in as ftp
|_   TYPE: ASCII
|_   No session bandwidth limit
|_   Session timeout in seconds is 300
|_   Control connection is plain text
|_   Data connections will be plain text
|_   At session startup, client count was 2
|_   vsFTPD 3.0.5 - secure, fast, stable
|_ End of status
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_ Can't get directory listing: TIMEOUT
22/tcp    open  ssh          OpenSSH 8.9p1 Ubuntu 3 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|_   256 21:ad:bf:e2:7c:62:18:0d:5c:94:23:6f:5b:10:5d:12 (ECDSA)
|_   256 17:d8:f4:63:63:b2:fe:38:7b:d0:12:b0:71:1a:ab:70 (ED25519)
80/tcp    open  http         Apache httpd 2.4.52 ((Ubuntu))
|_ http-robots.txt: 1 disallowed entry
|_ /
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Travel Blog
443/tcp   closed https
8080/tcp   closed http-proxy
MAC Address: 08:00:27:05:1D:46 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Aggressive OS guesses: Linux 5.0 - 5.14 (98%), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3) (98%), Linux 4.15 - 5.19 (94%), OpenWrt 21.02 (Linux 5.4) (94%), Linux 2.6.32 - 3.13 (93%), Linux 5.1 - 5.15 (93%), Linux 6.0 (93%), Linux 2.6.39 (93%), OpenWrt 22.03 (Linux 5.10) (93%), Linux 4.19 (92%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT ADDRESS
1 1.04 ms 192.168.56.2

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 57.85 seconds

(kalimainuserns@kali)-[~]
$
```

Analysing NMap Results

Open Ports and Services

- Summary of open/closed ports:
 - 21/tcp open ftp (vsftpd 3.0.5)
 - 22/tcp open ssh (OpenSSH 8.9p1)
 - 80/tcp open http (Apache 2.4.52)
 - 20/tcp, 443/tcp, 8080/tcp are closed.

Service Version Enumeration

- FTP (vsftpd 3.0.5):

- Allows anonymous login (ftp-anon: Anonymous FTP login allowed).
 - No bandwidth limit, plain text login – insecure.
 - Attack vector: Try connecting via FTP with username anonymous, look for file upload vulnerabilities, sensitive data disclosure, or misconfigurations.
- SSH (OpenSSH 8.9p1):
 - Secure remote administration.
 - Usually not worth attacking directly unless a weak password, old version, or brute-force has potential.
- HTTP (Apache 2.4.52):
 - Port 80 is open, running Apache under Ubuntu.
 - The web application is likely hosted here.
 - Attack vector: Test for web vulnerabilities (SQLi, XSS, file upload, info disclosure).
 - Shows a “Travel Blog” in the HTTP title; robots.txt allows 1 disallowed entry (could hint at hidden paths).

Recommended Attack Paths

- FTP (Port 21):
 - Use anonymous login to see if sensitive files are accessible/downloadable.
 - Check for directory traversal or misconfigured permissions.
- Web (Port 80):
 - Browse <http://192.168.56.2> to review the web application.
 - Run Dirb/Dirbuster for hidden directories.
 - Use Burp Suite, OWASP ZAP for vulnerability scanning (XSS, SQLi, insecure auth, etc.).
- SSH:
 - Only pursue if we can find credentials elsewhere (e.g., from FTP or web).
- Robots.txt:
 - Check <http://192.168.56.2/robots.txt> for restricted directories that might contain sensitive info.

Planning Next Steps

Document versions:

- Published vulnerabilities or exploits for
 - vsftpd 3.0.5
 - [OpenSSH 8.9p1](#)
 - [Apache 2.4.52](#)
- Focus on web application pentesting and possible abuses via FTP.
- Less emphasis on brute-forcing SSH unless login/password hints are found from other services.

Nmap output tells us which services to target, what potential weaknesses exist, and lets us plan our next moves – attack FTP with anonymous login, enumerate and attack the web site, investigate any sensitive files or directories, and note technology for future steps with Dirb, Burp, and ZAP.

Directory Brute-Forcing with Dirb or Dirbuster

Purpose: Find hidden directories or files that are not linked visibly on the web app.

The nmap scan revealed port 80 (HTTP) running Apache 2.4.52 is open, which means a web server is hosting content on the target. The purpose of web-based pentesting is to discover all possible web entry points, including hidden files and directories.

Directory Brute-Forcing (Dirb or Dirbuster) is a natural second step after nmap because:

- Nmap told us that web services are running (port 80, Apache)
- We want to find every possible “hidden” web source, admin page, config file or upload script exposed on the server
- These are prime targets for web vulnerabilities (XSS, authentication bypass, SQLi)
- We could attack FTP after the nmap (anonymous login allowed). Directory brute forcing is often prioritised in CTF (Capture the Flag) and analysis because it reveals web app structure - FTP exploration could be performed in parallel or shortly after a holistic approach.

Using Dirb

Run dirb against web root:

```
dirb http://192.168.56.2
```

```
kali linux [Running]

File Actions Edit View Help
(kalimainusers@kali)~$ dirb http://192.168.56.2

DIRB v2.22
By The Dark Raver

START_TIME: Sat Sep 20 02:20:00 2025
URL_BASE: http://192.168.56.2/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

--- Scanning URL: http://192.168.56.2/ ---

=> DIRECTORY: http://192.168.56.2/css/
=> DIRECTORY: http://192.168.56.2/images/
+ http://192.168.56.2/index.html (CODE:200|SIZE:2683)
=> DIRECTORY: http://192.168.56.2/js/
=> DIRECTORY: http://192.168.56.2/pages/
+ http://192.168.56.2/robots.txt (CODE:200|SIZE:71)
+ http://192.168.56.2/server-status (CODE:403|SIZE:277)

--- Entering directory: http://192.168.56.2/css/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/images/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/js/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/pages/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

END_TIME: Sat Sep 20 02:20:05 2025
DOWNLOADED: 4612 - FOUND: 3
```

Dirb uses its default wordlist (`/usr/share/dirb/wordlists/common.txt`). We may specify other wordlists for deeper scans.

Outputs directories/files found.

Why Directory Brute-Forcing Matters After Nmap

Nmap shows HTTP is running; brute-forcing directories immediately reveals the application's secret structure, giving us more targets for step 3 (manual exploitation, Burp Suite, ZAP testing).

We can often find `/admin`, `/config`, `/uploads`, which may not be linked from the main interface but still exist and could be vulnerable.

Example Workflow

- Nmap: Shows web service is active (port 80 open).
- Dirbuster/Dirb: Find hidden paths; `/admin` or `/uploads`.
- Browse new paths: Manual or automated testing with web scanners, attack found directories.
- Parallel FTP attack: Since anonymous login is allowed on port 21, check for sensitive files and potential credential leaks.

We used Dirb/Dirbuster after nmap because nmap confirms an HTTP service; directory brute-forcing systematically reveals the web server's structure for deeper attacks. Wordlists are vital – they're directories and file names that Dirb/Dirbuster tries in HTTP requests. The combo of nmap ("what's running?"), then brute-forcing ("what can I reach?"), is the classic pentest workflow.

Analysing the result of Dirb

The Dirb scan results indicate several discoverable and "listable" directories on the target web server.

Scanned with wordlist `/usr/share/dirb/wordlists/common.txt` against `http://192.168.56.2/`.

Directly discovered directories/files:

- `/css/`
- `/images/`

```
kali linux [Running]

(kalimainusersn@kali) ~
$ dirb http://192.168.56.2

DIRB v2.22
By The Dark Raver

START TIME: Sat Sep 20 02:20:00 2025
URL_BASE: http://192.168.56.2/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

GENERATED WORDS: 4612

--- Scanning URL: http://192.168.56.2/ ---

=> DIRECTORY: http://192.168.56.2/css/
=> DIRECTORY: http://192.168.56.2/images/
+ http://192.168.56.2/index.html (CODE:200|SIZE:2683)
=> DIRECTORY: http://192.168.56.2/js/
=> DIRECTORY: http://192.168.56.2/pages/
+ http://192.168.56.2/robots.txt (CODE:200|SIZE:71)
+ http://192.168.56.2/server-status (CODE:403|SIZE:277)

--- Entering directory: http://192.168.56.2/css/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/images/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/js/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/pages/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

END TIME: Sat Sep 20 02:20:05 2025
DOWNLOADED: 4612 - FOUND: 3
```

- /js/
- /pages/
- /index.html (main page)
- /robots.txt
- /server-status (potentially status or config info)

LISTABLE Directory Warning:

It means Directories like /css/, /images/, /js/, /pages/ are “listable” – their contents can be viewed in a browser, showing all files inside. This is often insecure, as attackers can see resources that should be hidden.

Planning Our Next steps

- Dirb shows which web directories and files are exposed. Now, let's visit each of them, enumerate their content, and plan targeted exploitation using Burp Suite, ZAP, or manual browser actions.
- Link findings with what we know from Nmap; e.g., see if any file in /pages/ or similar leaks info that could help us attack FTP, SSH, or escalate privileges.
- This Dirb output shows that our scan of `http://192.168.56.2/` found several directories (`/css/`, `/images/`, `/js/`, `/pages/`) and files (`/index.html`, `/robots.txt`, `/server-status`).
- The "LISTABLE" warning means these directories allow directory listing, so their contents are visible to anyone browsing that path in a browser.

What this result means

- Each found directory could contain files or scripts not normally linked from the main site, which may be poorly secured or provide clues about app logic.
- Directory listing is often a misconfiguration; it may expose sensitive files, scripts, or backups that should not be publicly viewable.
- Files like `/robots.txt` may point to locations meant to be hidden from search engines, while `/server-status` can leak server information.

Next steps

- Manually visiting each directory in the browser and looking at the contents.
- Making note of all files revealed – checking for source code, config files, or anything unusual.

- Visiting `/robots.txt` for clues about additional sensitive or hidden areas.
- Visit `/server-status` to see if info about the server or running applications is exposed.
- Use a larger wordlist (e.g., `/usr/share/wordlists/dirb/big.txt`) if we want to discover more obscure directories or files.

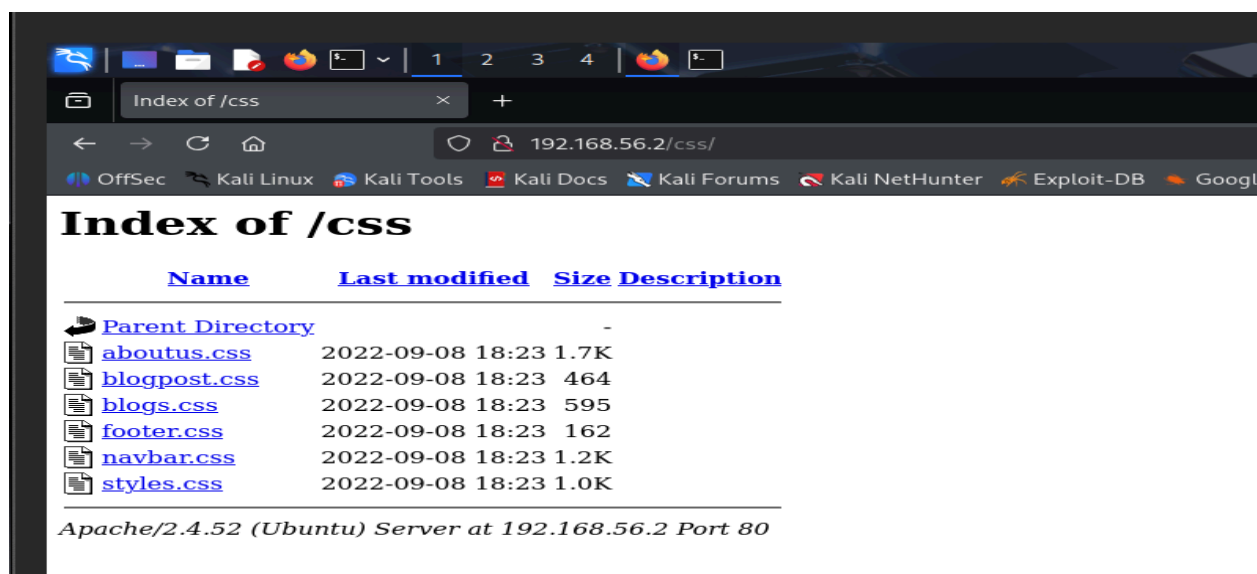
Linking with nmap results

We use Dirb because nmap shows that HTTP (port 80) is open; Dirb targets the web service exposed on that port.

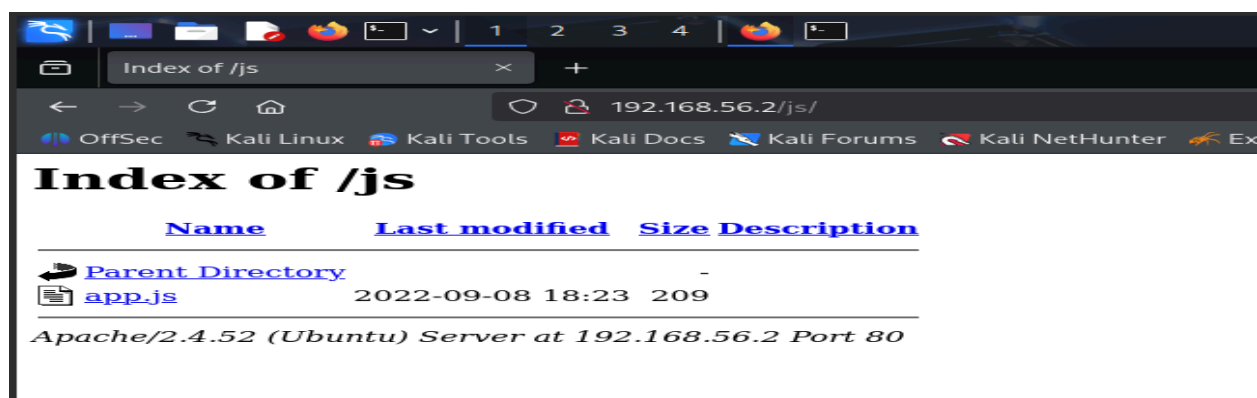
Findings from Dirb may directly influence later attacks with Burp Suite, ZAP, or manual testing—any discovered script or page is a candidate for further investigation for vulnerabilities.

Manual Exploration of Exposed Targets

- <http://192.168.56.2/css>

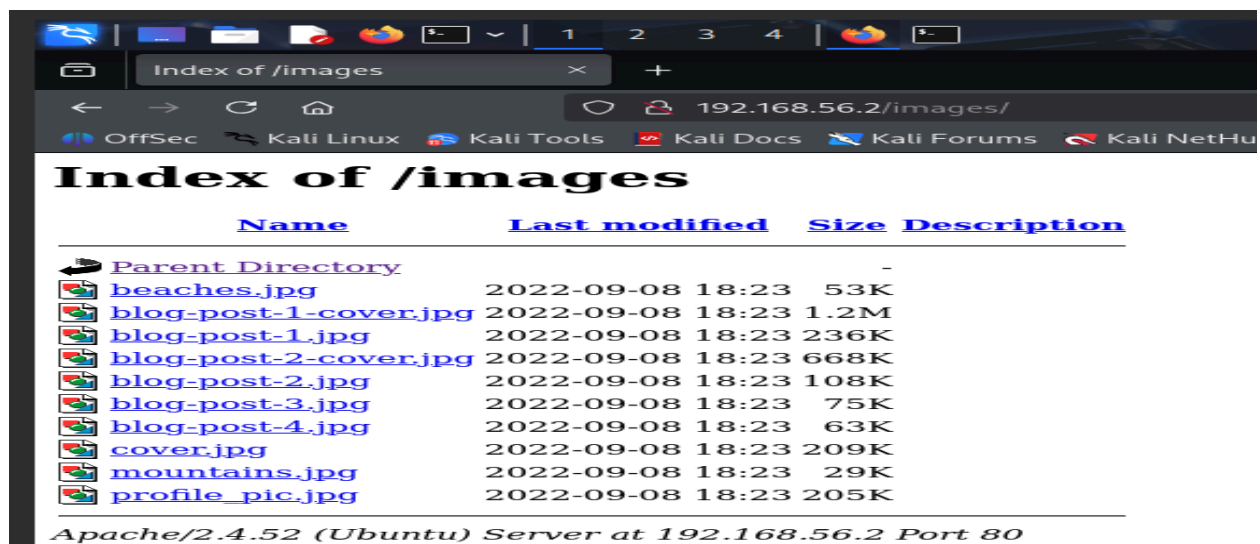


- <http://192.168.56.2/js/>

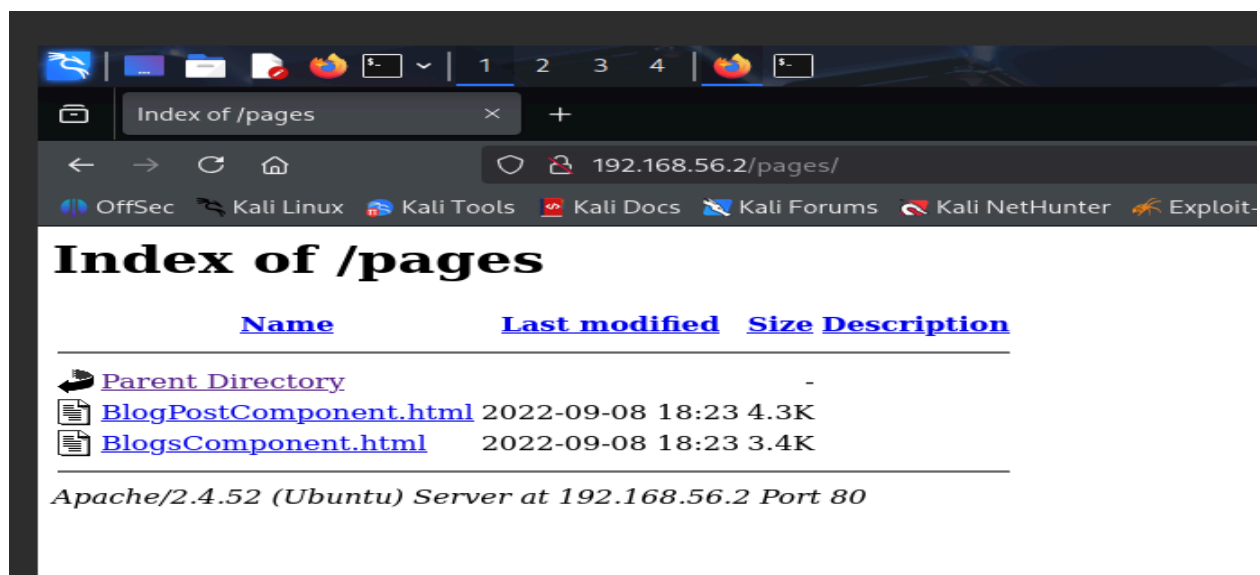


w

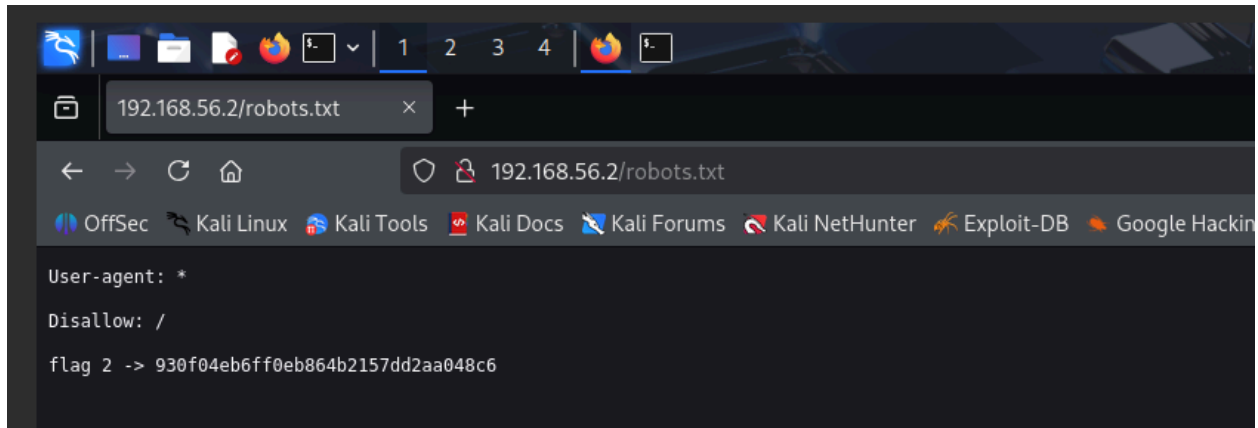
- <http://192.168.56.2/images>



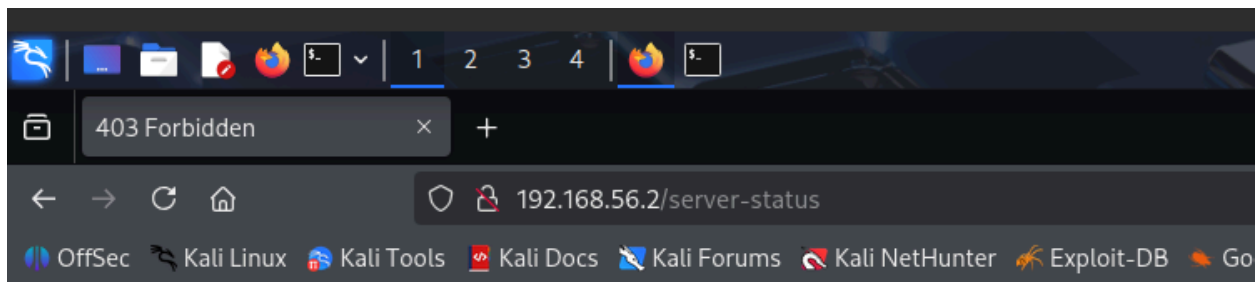
- <http://192.168.56.2/pages/>



- <http://192.168.56.2/robots.txt>



- <http://192.168.56.2/server-status>

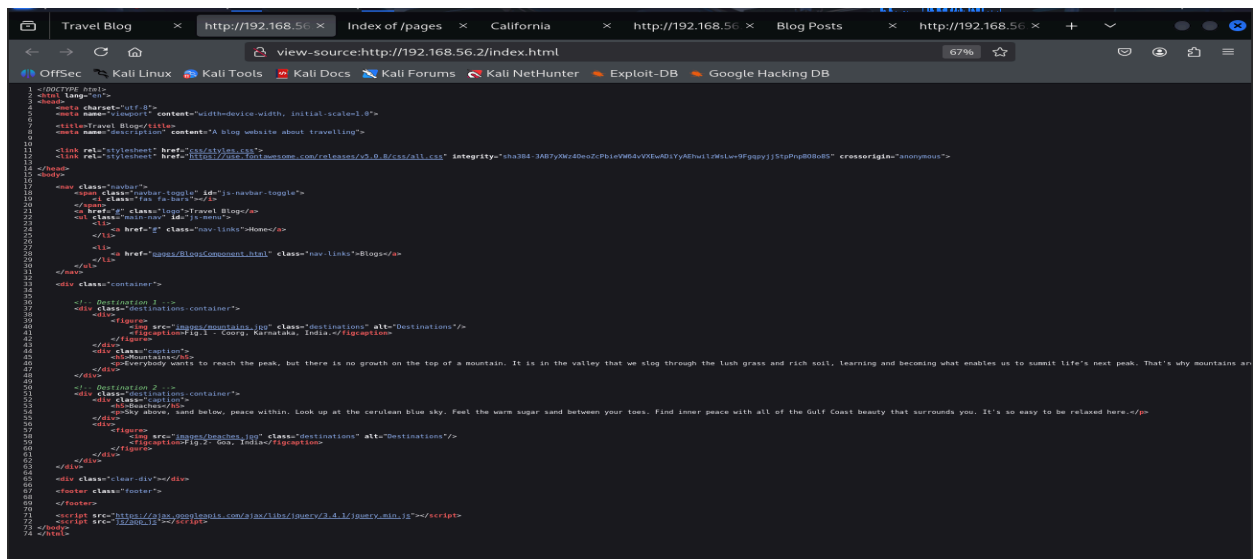


Forbidden

You don't have permission to access this resource.

Apache/2.4.52 (Ubuntu) Server at 192.168.56.2 Port 80

Reviewing Source Code



- Exploring if we can POST/GET to new endpoints referenced by app.js.
- Try directly accessing files with similar names (guessing), e.g., /pages/admin.html

Our Findings

- Flag

```
Disallow: /

flag 2 -> 930f04eb6ff0eb864b2157dd2aa048c6
```

```
49 <div class="clear-div"></div>
50
51 <footer class="footer">
52   <!-- flag 3 -> b35e7489cf89d4188c85d921a2f79821 -->
53 </footer>
54
55 <script src="._/js/app.js"></script>
56 </body>
57 </html>
```

- Files are listable and contents publicly exposed (a critical finding)
- Document 403 on /server-status as a sign of improved, but not perfect, hardening

Summary:

- We are mapping actual site content, confirming what Dirb found, revealing sensitive data (flag 2), and preparing for more advanced attacks by analyzing page source and dynamic web requests.
- This is the core of how enumeration moves into vulnerability identification and exploitation

Analysing /pages/BlogsComponent.html

Observations

- The code is mostly static HTML with no visible forms, user-input fields, or backend interaction.
- Uses client-side JavaScript only for navbar toggling (from /js/app.js).
- No embedded credentials, API keys, or comments with sensitive information.

Potential issues

- Reveals static blog post data—no direct exploit unless JavaScript or other dynamically referenced resources are hidden.

Next Action:

- Check for hidden JavaScript or AJAX calls (not seen here).
- Use Burp/ZAP to monitor any additional requests as we interact with the live page.

Analysing /pages/BlogPostComponent.html

Observations:

- As discovered in previous section, this file does contain a hidden HTML comment at the bottom:
`<!-- flag 3 -> b35e7489fc894b188c85d921a2f79821 -->`
- This is a CTF "flag"—capture and record for our project as a discovered proof.
- No forms, input boxes, or dynamic elements seen.

Potential issues:

- In a real application, hidden comments with sensitive data will become a vulnerability. Here, it's intentional for the exercise.

Action:

- Always look for HTML comments with sensitive info in every page's source.
- Record this flag as proof of access.

Analysing `/index.html`

Observations:

- Basic static front page.
- Links to the blogs component via `href="pages/BlogsComponent.html"`.
- Includes images and a navigation bar, no form fields or dynamic actions in the HTML.
- No plaintext credentials or sensitive comments found.

Potential issues:

- No clear security issue in this static code.
- Ensure to check if any JavaScript referenced loads secondary data (we can review `/js/app.js` again).

Action:

- Look for hidden fields, unlinked content, or dynamically loaded scripts.

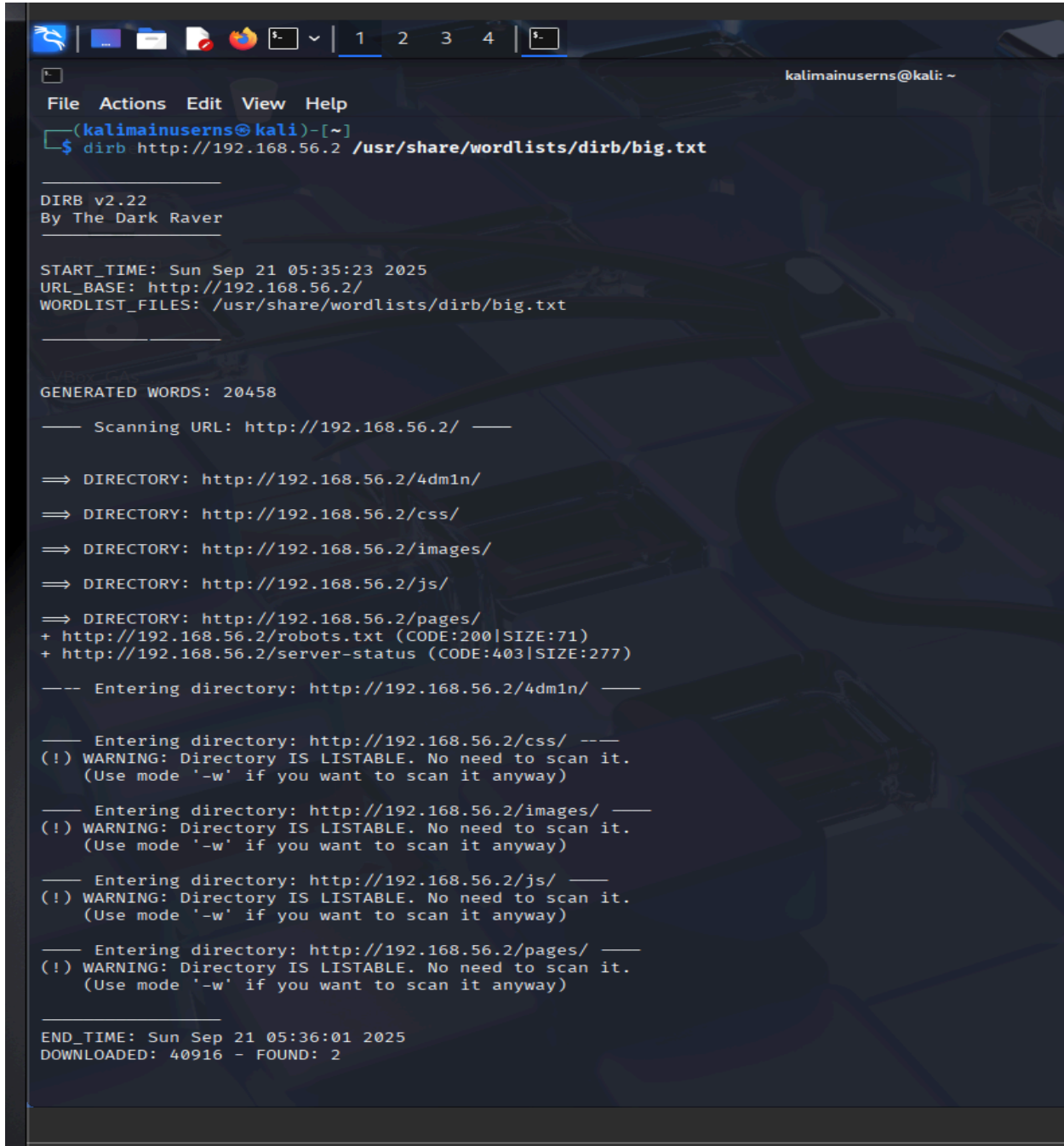
General Next Steps

- No credentials or secrets are visible other than the intended CTF flag.
- Main potential vulnerabilities would only appear if there were form fields (for XSS, SQLi) or AJAX/dynamic calls in scripts—none detected in the code provided.

For further testing:

- Use dirb with `big.txt`
- Proxy browser traffic through Burp Suite or OWASP ZAP while browsing these pages to intercept AJAX calls, hidden resources, or API endpoints that don't appear in the static source.
- Brute-force or guess other file and page names based on patterns we see (e.g., there may be additional Component files or similar hidden admin pages).

Testing with dirb big.txt



```
File Actions Edit View Help
(kalimainusersn@kali)-[~]
$ dirb http://192.168.56.2 /usr/share/wordlists/dirb/big.txt

DIRB v2.22
By The Dark Raver

START_TIME: Sun Sep 21 05:35:23 2025
URL_BASE: http://192.168.56.2/
WORDLIST_FILES: /usr/share/wordlists/dirb/big.txt

GENERATED WORDS: 20458

--- Scanning URL: http://192.168.56.2/ ---

=> DIRECTORY: http://192.168.56.2/4dm1n/
=> DIRECTORY: http://192.168.56.2/css/
=> DIRECTORY: http://192.168.56.2/images/
=> DIRECTORY: http://192.168.56.2/js/
=> DIRECTORY: http://192.168.56.2/pages/
+ http://192.168.56.2/robots.txt (CODE:200|SIZE:71)
+ http://192.168.56.2/server-status (CODE:403|SIZE:277)

--- Entering directory: http://192.168.56.2/4dm1n/ ---

--- Entering directory: http://192.168.56.2/css/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/images/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

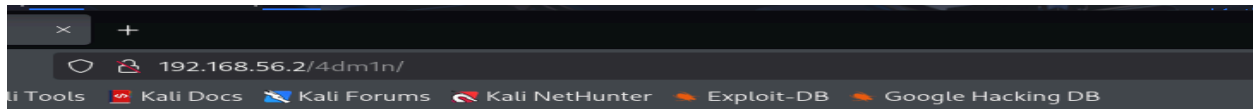
--- Entering directory: http://192.168.56.2/js/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

--- Entering directory: http://192.168.56.2/pages/ ---
(!) WARNING: Directory IS LISTABLE. No need to scan it.
(Use mode '-w' if you want to scan it anyway)

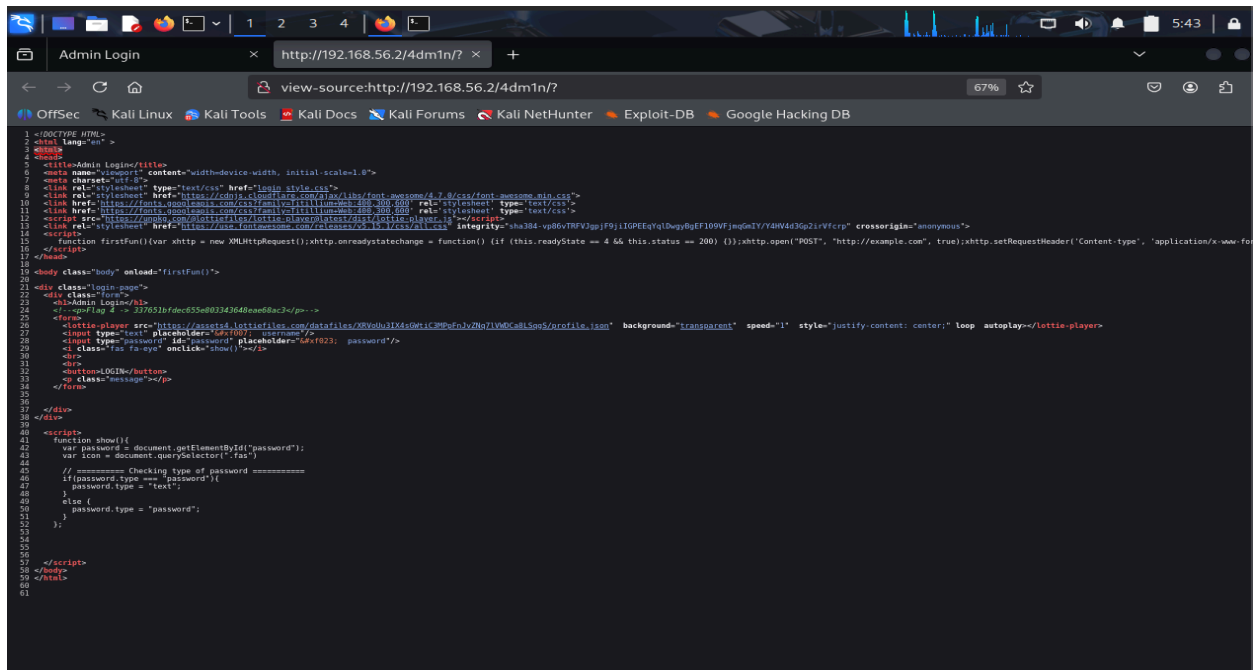
END_TIME: Sun Sep 21 05:36:01 2025
DOWNLOADED: 40916 - FOUND: 2
```

We can see one new entry: /4dm1n

Accessing <http://192.168.56.2/4dm1n>



This is the admin page. Next we go to view page source. Screenshot



below.

Analysing above output

Dirb Output (big.txt wordlist)

- With a larger wordlist, Dirb discovered /4dm1n/ as a new directory not found before.
- This suggests the admin login page is somewhat “hidden-by-obscurity,” as a basic wordlist wouldn’t uncover it.
- This is a common way to hide sensitive functionality, but not secure against determined attackers.

/4dm1n/ Page and Source

- The page is a basic “Admin Login” form for username and password.
- Source Code Review:
- There’s a flag:
`<!-- flag 4 -> 3f257b16cf655c0033d643e68ae68ac3 -->` in an HTML comment.
- No credentials are visible, but the login form is present.
- Simple client-side JS lets us show/hide passwords, but there’s no security vulnerability in that itself.

3. Next Steps

- Record the new flag discovered in the HTML comment:
`flag 4 -> 3f257b16cf655c0033d643e68ae68ac3`


```

19 <body class="body" onload="firstRun()">
20
21 <div class="login-page">
22   <div class="form">
23     <h1>Admin Login</h1>
24     <!--<p>Flag 4 -> 337651bfdec655e803343648eae68ac3</p>-->
25     <form>
26       <lottie-player src="https://assets4.lottiefiles.com/datafiles/XRVoUu3IX4sGWtiC
27       <input type="text" placeholder="&#xf007; username"/>
28       <input type="password" id="password" placeholder="&#xf023; password"/>
29       <i class="fas fa-eye" onclick="show()"></i>

```

- Use **Burp Suite** or **ZAP** (proxies) to intercept our login attempts and test for:
 - Error messages
 - Weak or default credential acceptance
 - SQL injection responses (try simple payloads)
 - Parameter tampering
- If the server returns different status codes or error pages, follow up with further enumeration and payloads.
- If **brute-forcing** or **injection** allows admin login, document the process and what data, pages, or admin power is gained.
- **Try common credentials** (admin:admin, admin:password, etc.) or payload-based bypasses.

Summary:

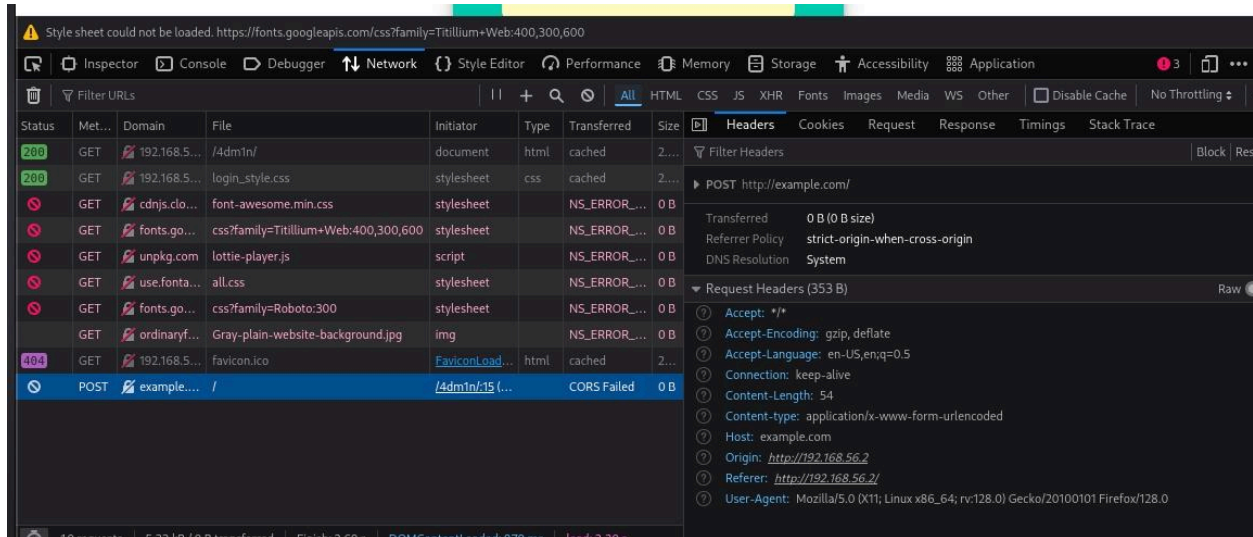
- If we can access the backend (for example, with Burp Suite), we can:
 - Try **brute-forcing credentials** since the admin page is now found.
 - Test for **SQL injection**—try submitting ' OR '1'='1 or similar in either username or password to see if we can bypass authentication.
 - **Inspect the login** functionality using **Burp Suite/ZAP** to analyze requests, parameters, and responses for hints of vulnerabilities.

Brute Force Attack on admin login page

Confirm Form Details

From the HTML code:

- *Login URL*: `http://192.168.56.2/4dm1n/`
- *Fields*: username and password
- *Method*: Most typical login forms use POST. (The code shows an HTML `<form>` without an explicit method, but most browsers default to POST unless specified.)
- Manually test the login with an incorrect username/password. Note the error or response text (e.g., "Invalid username or password", "Login failed").
- This string will help Hydra know when a login attempt fails.



This is the screenshot from the developer tools → network tab after entering username and password.

Analysing the Network response after entering the login credentials

The screenshot of the browser's network tab shows that when the login fails, there isn't any error message returned to the web page, and the POST request goes to the dummy endpoint `http://example.com/` (not the vulnerable VM) due to placeholder JavaScript code.

Key Points from the Developer Tools

- The actual login form (when submitted) triggers a POST to `http://example.com/` (seen in the request headers).
- This is not the real backend endpoint—it's hardcoded as a placeholder in the site's JavaScript for demonstration/testing.
- No useful response, no error message, no indicator of "success" or "failure" from the web server.

What This Means for Hydra

- Hydra needs to check the real backend login endpoint—not just this dummy request.
- Since the login form's backend doesn't appear to be implemented, brute-forcing with Hydra on `http://192.168.56.2/4dm1n/` will likely not work unless the actual server app processes POST requests there and returns meaningful responses.

Actions to Take

Confirm Backend Login Logic

- Try submitting the form with real values: Does the page ever change, reload, or display a message?
- Try submitting while inspecting the network tab—do any requests go to `/4dm1n/` or another endpoint on the actual VM IP?

If /4dm1n/ Accepts POST:

- Hydra can be set up to brute-force. In the absence of a failure string, use HTTP status codes by setting the "fail" string to something generic or use Hydra's option to check for differing lengths/responses.

Example command:

```
As hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.56.2  
http-post-form "/4dm1n/:username=^USER^&password=^PASS^:S=200"
```

(Here, S=200 just looks for a successful HTTP response (change if needed)).

Run and Monitor

- Run Hydra and watch for successful logins.
- If successful, Hydra will display the username and password.
- Log in manually to confirm access and search for new flags, sensitive data, or admin functionality.

Summary:

- Use Hydra with the admin login URL, fields, and ready dictionary.
- Adjust "F=Login failed" with the actual failure response.
- On a successful brute-force, use discovered credentials for further testing.

If Login Not Actually Implemented:

- The login form may be intentionally nonfunctional for this exercise.
- Focus on other vulnerabilities (e.g., exposed flags, directory listings, FTP access, or try for SQL injection if the form has any backend logic

Check Open Port FTP

Step 1: Connect to FTP Server

Run:

```
ftp 192.168.56.2
```

We see a message `Connected to 192.168.56.2.`

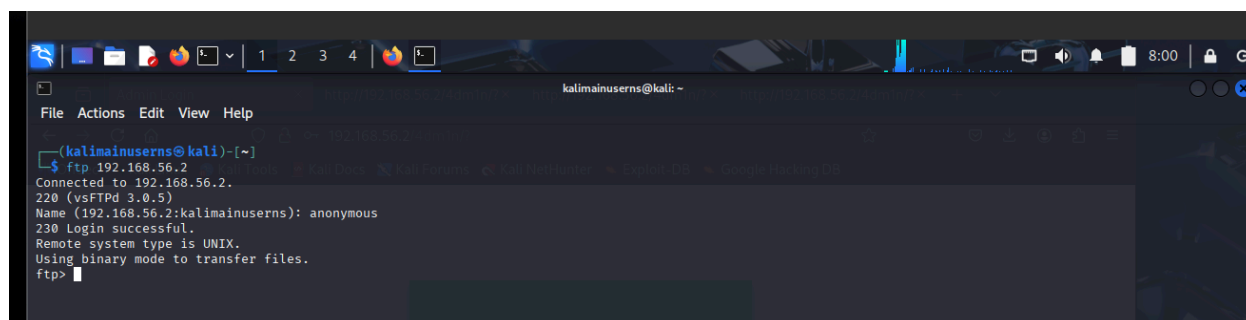
Step 2: Login Anonymously

When prompted for username, enter:

```
anonymous
```

When prompted for password, enter any string or the email address (just press Enter or type anything).

We should see a `login success` message.



```
kalimainusersn@kali: ~  
File Actions Edit View Help  
(kalimainusersn@kali)-[~]  
$ ftp 192.168.56.2  
Connected to 192.168.56.2.  
220 (vsFTPd 3.0.5)  
Name (192.168.56.2:kali): anonymous  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp>
```

Step 3: List Files and Directories

Run:

```
ls
```

This lists files and directories accessible with anonymous login.

If ls fails, try:

```
dir
```

Note down all files and directories listed.

Note: Sometimes we see an `Entering Extended Passive Mode` message. It indicates that the FTP client and server are negotiating passive mode for data transfer. In Extended Passive Mode (EPSV), the server tells the client which port to use for the data channel, used to transfer directory listings and files. EPSV is compatible with both IPv4 and IPv6 and works better through firewalls and NATs compared to traditional active FTP mode. The client then establishes a data connection to the server's specified port rather than the server connecting back to the client.

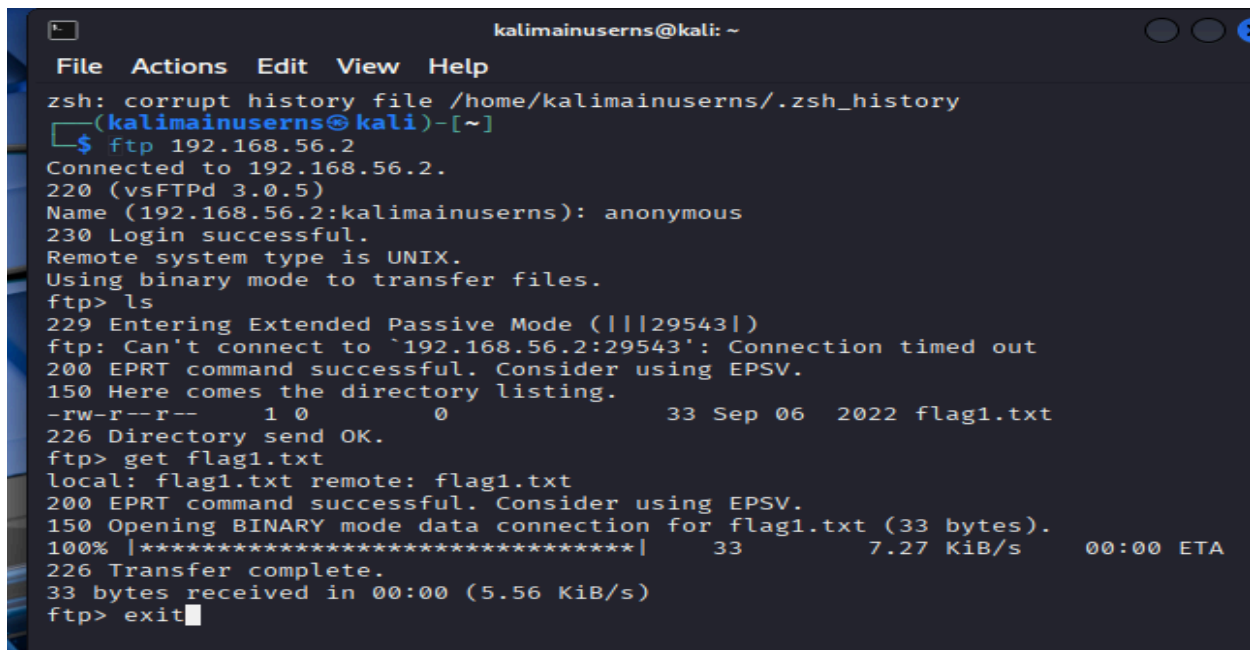
Step 4: Navigate and Download Files

Change directory:

```
cd <directory_name>
```

Download a file:

```
get <filename>
```



```
kalimainusersns@kali: ~  
File Actions Edit View Help  
zsh: corrupt history file /home/kalimainusersns/.zsh_history  
(kalimainusersns@kali)-[~]  
$ ftp 192.168.56.2  
Connected to 192.168.56.2.  
220 (vsFTPD 3.0.5)  
Name (192.168.56.2:kalimainusersns): anonymous  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp> ls  
229 Entering Extended Passive Mode (|||29543|)  
ftp: Can't connect to `192.168.56.2:29543': Connection timed out  
200 EPRT command successful. Consider using EPSV.  
150 Here comes the directory listing.  
-rw-r--r-- 1 0 0 33 Sep 06 2022 flag1.txt  
226 Directory send OK.  
ftp> get flag1.txt  
local: flag1.txt remote: flag1.txt  
200 EPRT command successful. Consider using EPSV.  
150 Opening BINARY mode data connection for flag1.txt (33 bytes).  
100% |*****| 33 7.27 KiB/s 00:00 ETA  
226 Transfer complete.  
33 bytes received in 00:00 (5.56 KiB/s)  
ftp> exit
```

Download all files (if many):

```
mget *
```

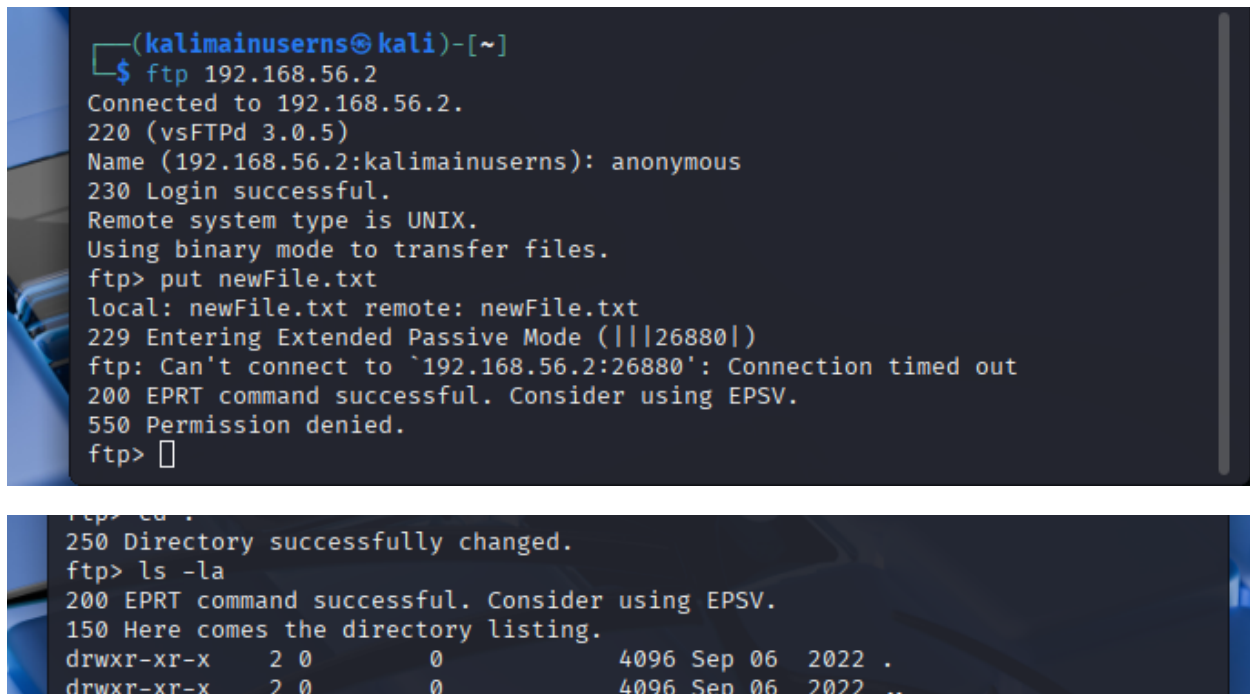
Save files locally for later analysis.

Step 5: Upload File (Test Write Permission)

Check if we can upload (write permissions):

```
put <local_test_file>
```

If upload succeeds, the server is misconfigured and vulnerable.



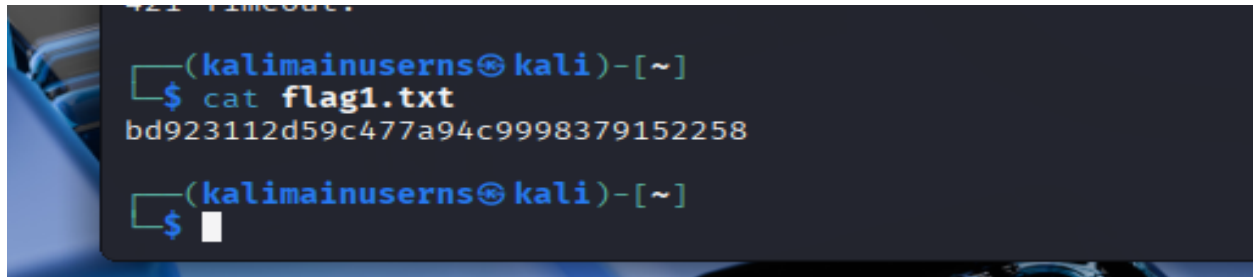
```
(kalimainusersn@kali)-[~]  
$ ftp 192.168.56.2  
Connected to 192.168.56.2.  
220 (vsFTPd 3.0.5)  
Name (192.168.56.2:kalimainusersn): anonymous  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp> put newFile.txt  
local: newFile.txt remote: newFile.txt  
229 Entering Extended Passive Mode (|||26880|)  
ftp: Can't connect to `192.168.56.2:26880': Connection timed out  
200 EPRT command successful. Consider using EPSV.  
550 Permission denied.  
ftp>  
  
ftp> cd .  
250 Directory successfully changed.  
ftp> ls -la  
200 EPRT command successful. Consider using EPSV.  
150 Here comes the directory listing.  
drwxr-xr-x  2 0      0          4096 Sep 06  2022 .  
drwxr-xr-x  2 0      0          4096 Sep 06  2022 ..
```

Step 6: Summary of Vulnerabilities

- *Anonymous login allowed*: Sign of weak FTP password policy.
- *Directory listing enabled*: Allows attackers to gather info on available files.
- *No Upload permission*: Anonymous users do NOT have write/upload permission on this FTP server. The 550 Permission denied error

confirms the server is configured to block uploads by users logged in as "anonymous."

- *Anonymous users DO have read/download permission*, since we were able to fetch and download existing files
- *Downloadable sensitive files*: Lets Look for config files, backups, credentials. After listing all hidden files we didn't find anything.

A terminal window with a dark background. The prompt is `(kalimainusersn@kali)-[~]`. The user enters `$ cat flag1.txt` and the output is `bd923112d59c477a94c9998379152258`. The prompt is then `(kalimainusersn@kali)-[~]` with a cursor on the next line.

```
(kalimainusersn@kali)-[~]  
$ cat flag1.txt  
bd923112d59c477a94c9998379152258  
  
(kalimainusersn@kali)-[~]  
$
```

Step 7: Exit FTP

Run:

`bye`

To quit the FTP session.

Burp Suite

Burp Suite Introduction

- Burp Suite is an integrated platform for performing security testing of web applications.
- It is commonly used by pentesters to intercept, inspect, and modify web traffic between their browser and a target web application.
- With Burp Suite, we can find and exploit vulnerabilities like XSS, SQL injection, broken authentication, and more.

Why Burp Suite?

- It acts as an intercepting proxy to capture and modify HTTP/HTTPS traffic.
- Burp Suite is called a proxy because its core function is to act as an HTTP proxy server that sits between the browser and the target website, allowing us to intercept, inspect, and modify the network traffic in real-time. This intermediary role enables security testers to analyze requests and responses, revealing how applications interact with servers and identifying potential vulnerabilities.
- Provides automated vulnerability scanning.
- Allows detailed manual testing of web requests/responses.
- Helps in identifying security flaws by observing web app behavior in real-time

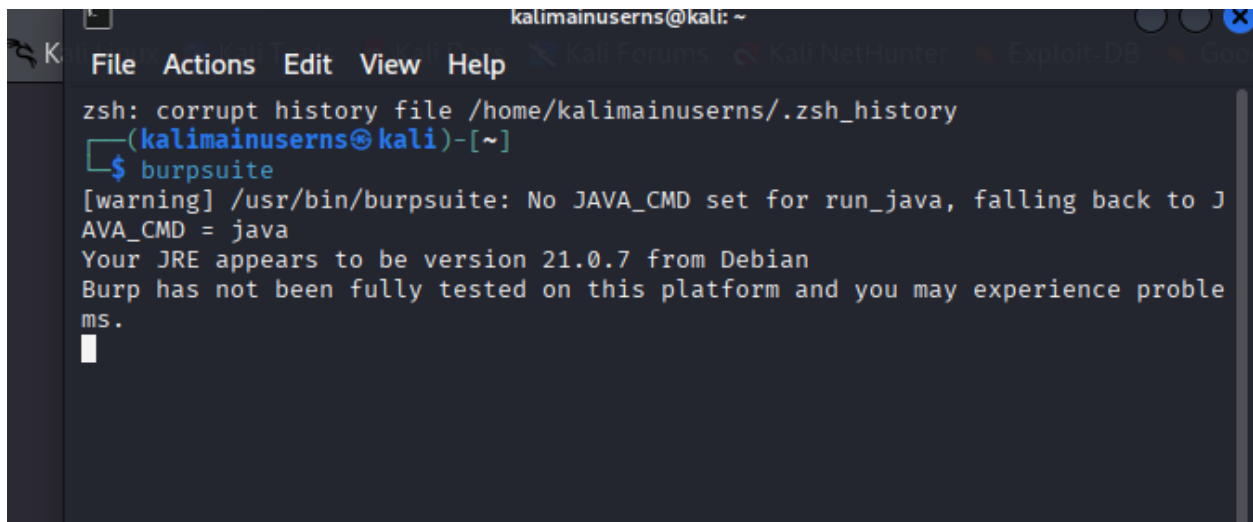
Where is it Used?

- Web application penetration testing engagement.
- Application security assessments.
- Ethical hacking CTF labs and real-world security audits.

Step-by-Step Guide to Set Up Burp Suite for Penetration Testing

Launch Burp Suite on Kali Linux

- Open a terminal or use the Kali Linux menu to find and start Burp Suite by typing:
`burpsuite`
- The Burp Suite application window will open. Choose Temporary Project > Start Burp.

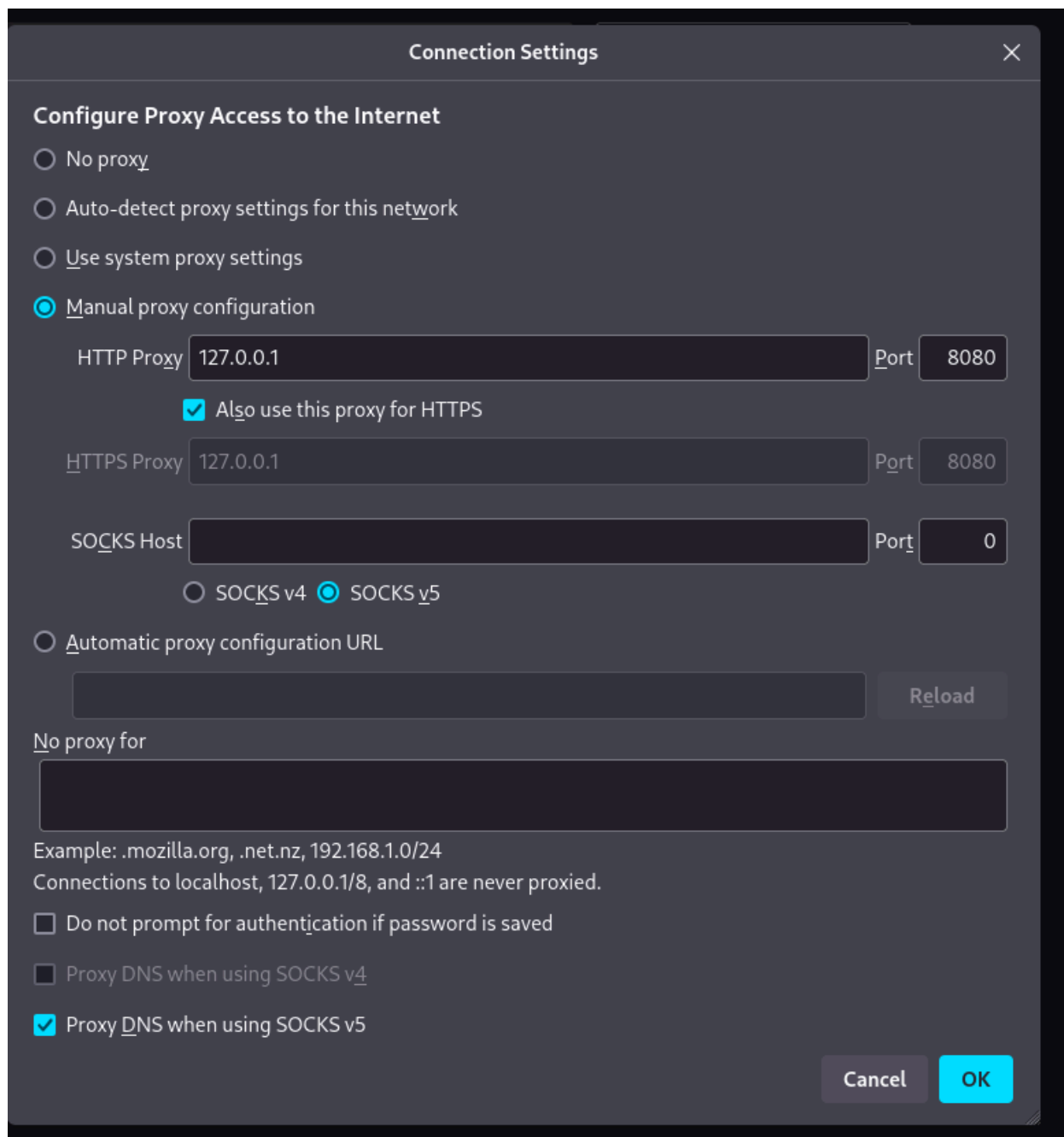


```
kalimainusersns@kali: ~  
File Actions Edit View Help  
zsh: corrupt history file /home/kalimainusersns/.zsh_history  
(kalimainusersns@kali)-[~]  
$ burpsuite  
[warning] /usr/bin/burpsuite: No JAVA_CMD set for run_java, falling back to J  
AVA_CMD = java  
Your JRE appears to be version 21.0.7 from Debian  
Burp has not been fully tested on this platform and you may experience proble  
ms.  
█
```

Configure Browser to Use Burp as Proxy

For Firefox (default browser in Kali):

- Open Firefox and go to Preferences.
- Scroll down to Network Settings.
- Click Settings... to open Connection Settings.
- Select Manual proxy configuration.



In HTTP Proxy, enter:

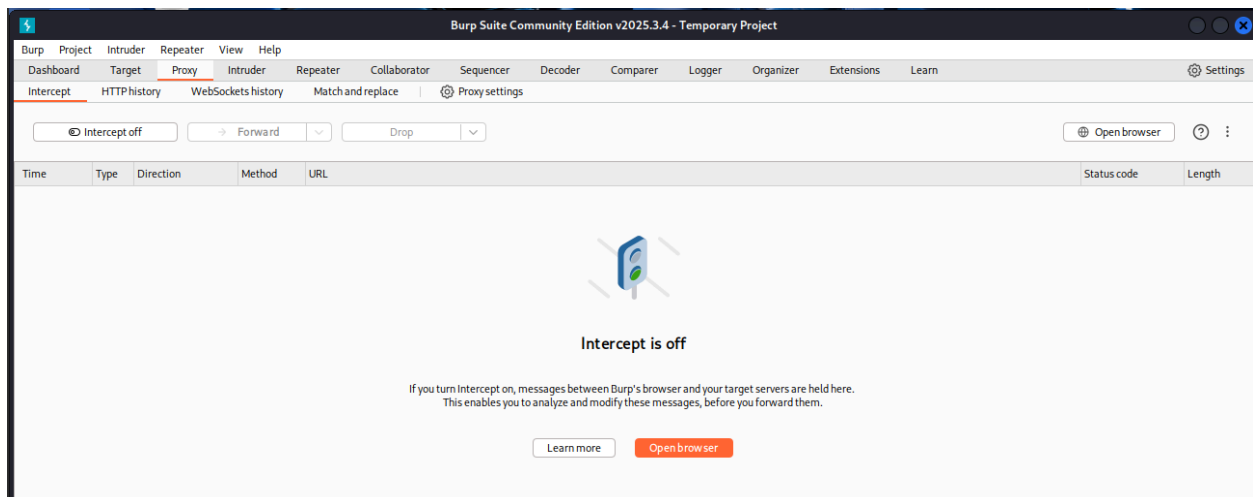
127.0.0.1

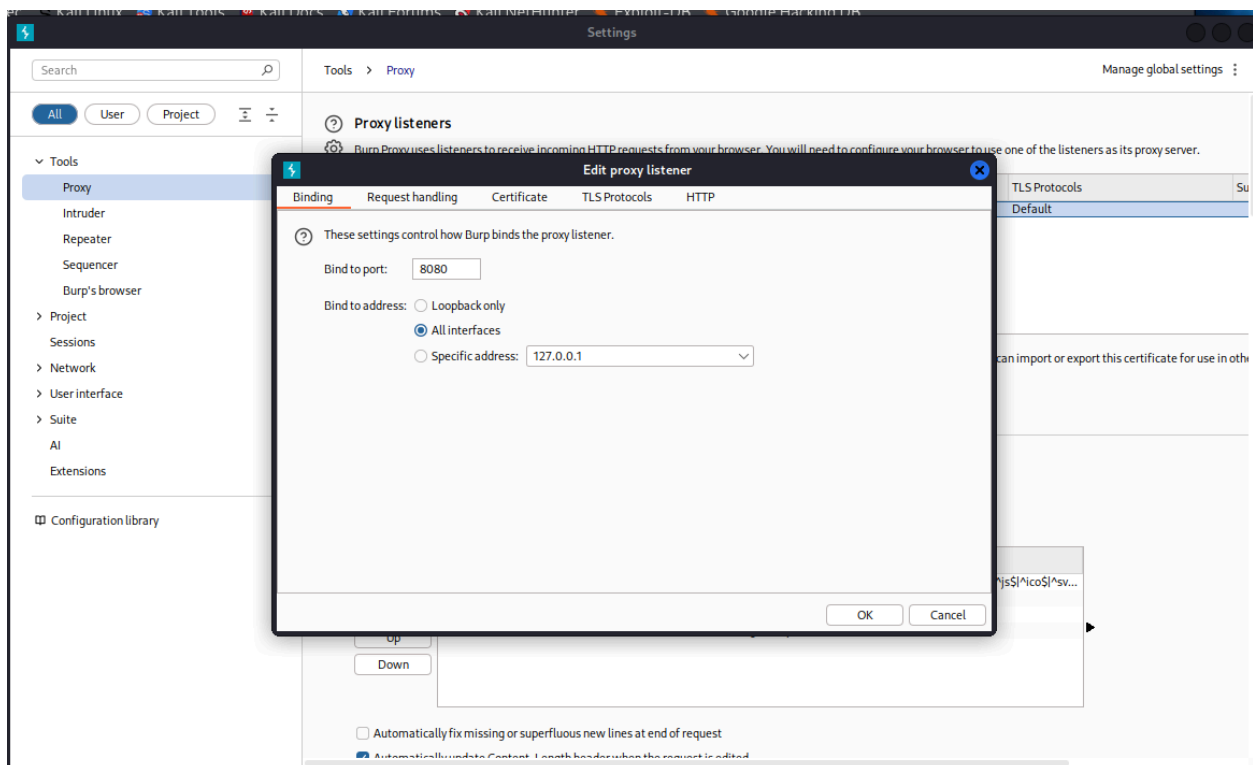
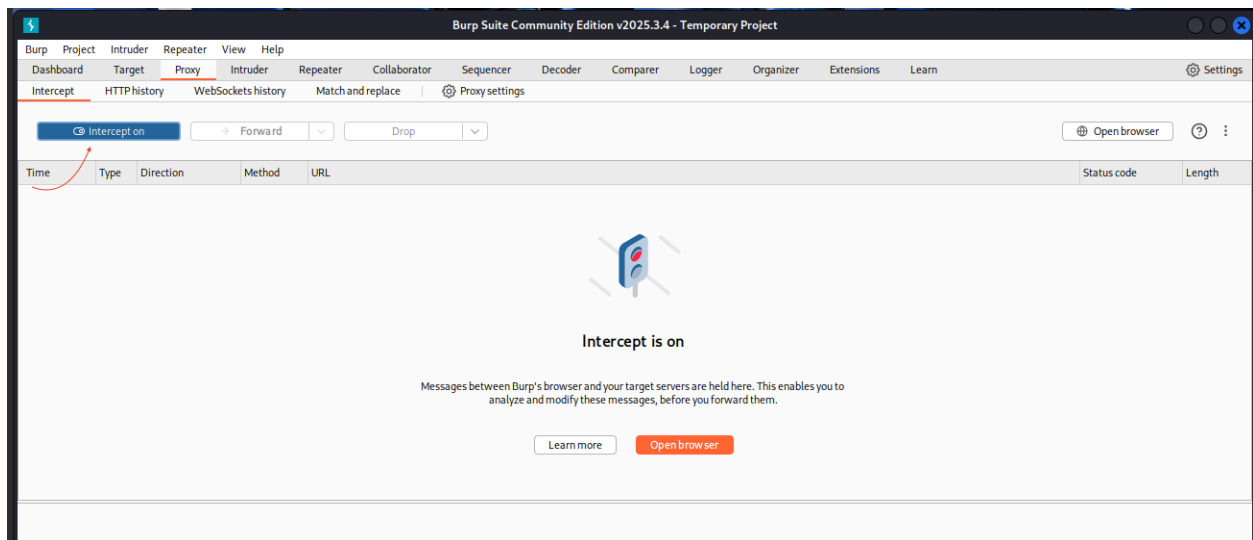
- In the Port field next to HTTP Proxy, enter:
8080
- Check the box Use this proxy server for all protocols.

- Click **OK** to save settings.

Verify Proxy Configuration

- In Burp Suite, navigate to the Proxy tab and make sure **Intercept** is turned **ON**.
- Open a webpage in Firefox (e.g., `http://192.168.56.2`) and watch Burp capture the request.
- If Burp intercepts the HTTP request, the proxy is correctly set up.





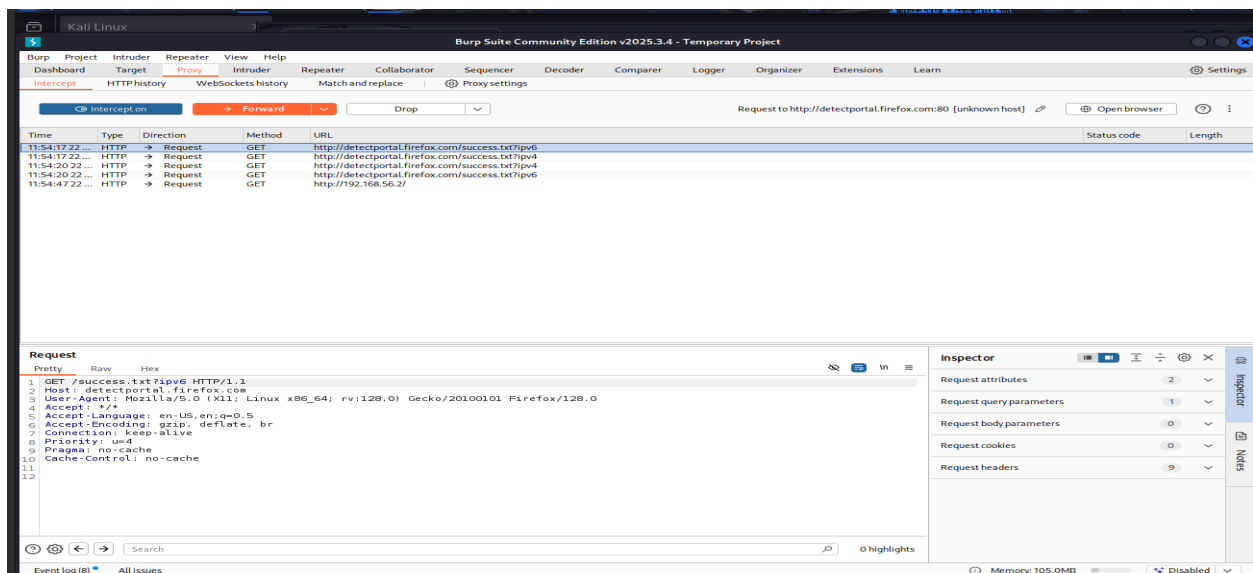
Notes for HTTPS Sites

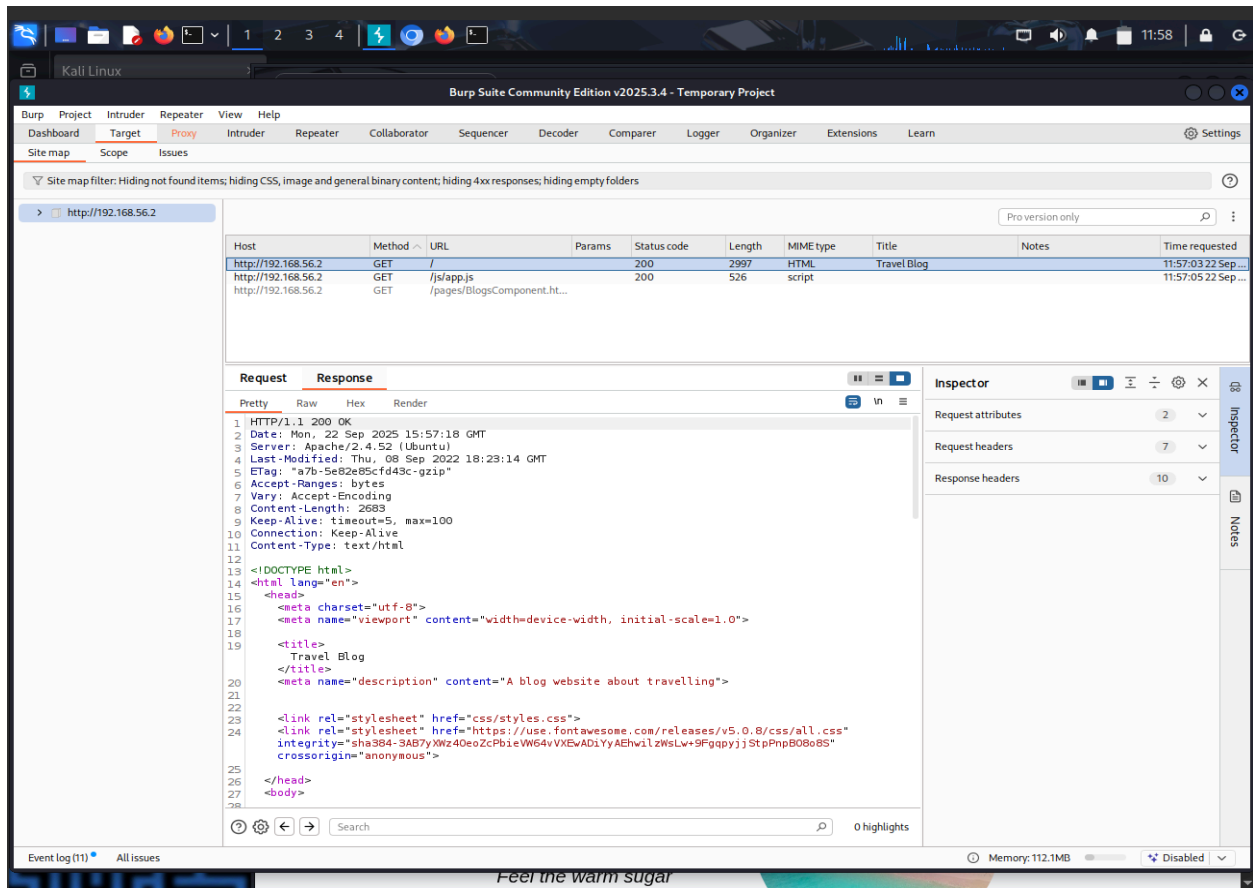
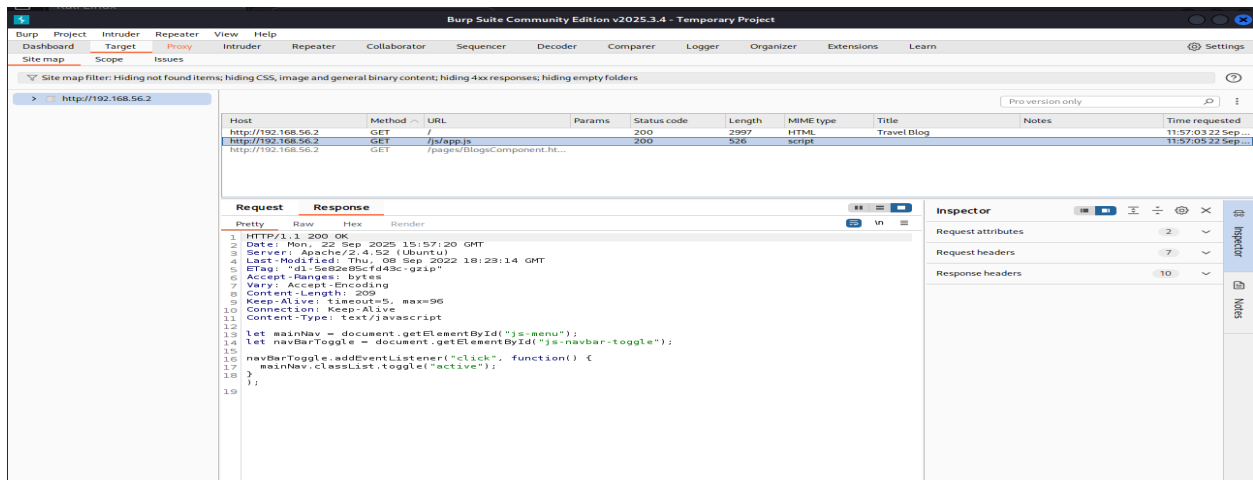
- For HTTPS traffic interception, install Burp's CA certificate in the browser to avoid SSL errors.

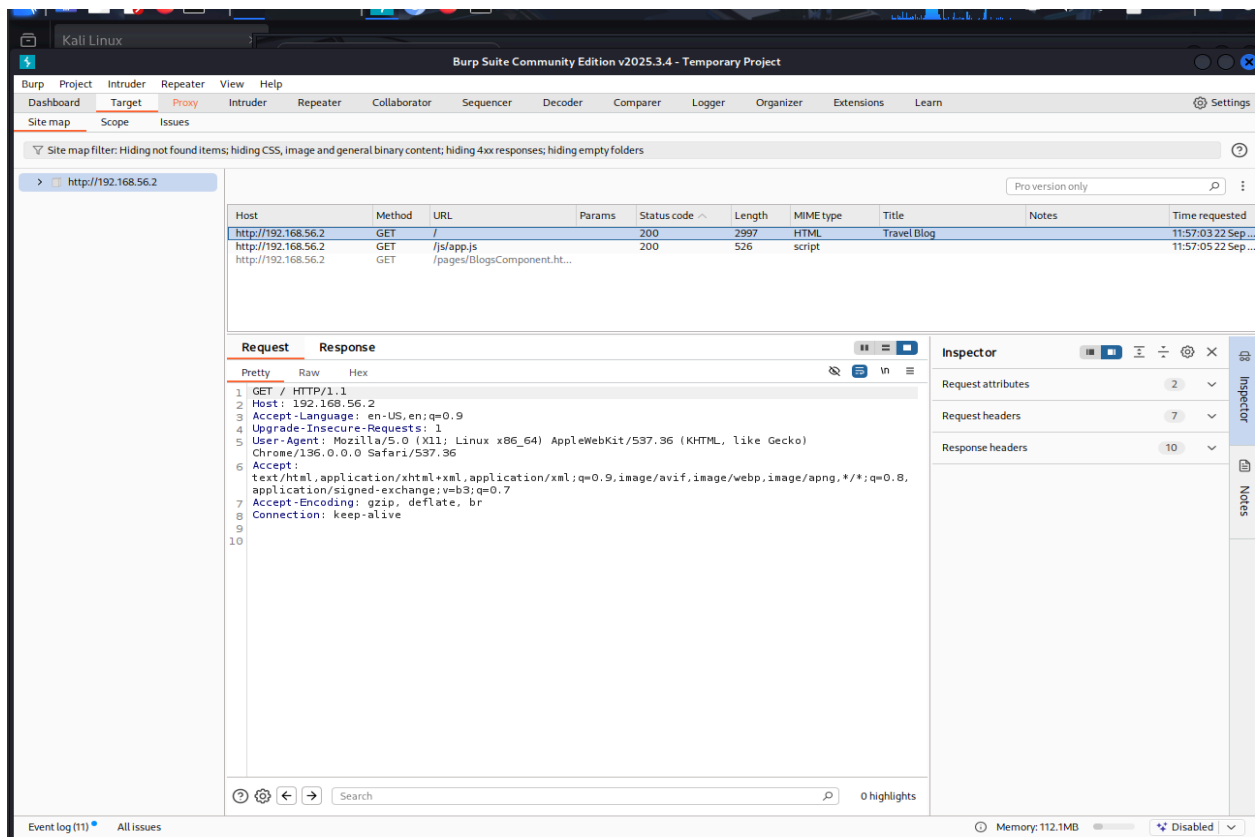
Note: For our local vulnerable VM on HTTP, this is not needed.

Intercept Traffic

- In Burp, go to **Proxy > Intercept** tab and ensure the intercept is ON.
- Browse to the target web application, for example <http://192.168.56.2>.
- Burp will pause HTTP requests and allow us to review and modify them before forwarding.







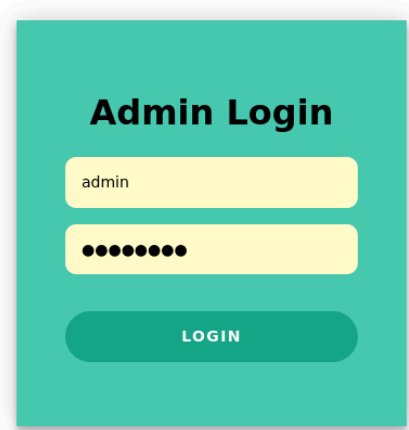
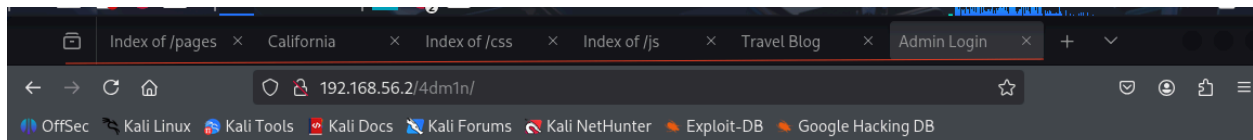
Manual Crawling

Burpe Suite Professional Edition comes with Crawler or Spider. Since we are using the Community edition we will attempt manual crawling Techniques.

Browse the Application Like a User

- With Burp running, visit all pages of the target vulnerable application in the browser.
- Manually click every link, menu item, and accessible feature.
- Fill out and submit every form with test data, trying different combinations.

Admin Page



- In the Burp Suit → Proxy tab - Inspect details of each page one by one.
- Enter the url in browser: `http://192.168.56.2/4dm1n`

Burp Suite Community Edition v2025.3.4 - Temporary Project

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to http://example.com:80 [unknown host] Open browser ?

Time	Type	Direction	Method	URL	Status code	Length
00:06:18...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv6		
00:06:19...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv4		
00:06:27...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv4		
00:06:27...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv6		
00:16:49...	HTTP	→ Request	GET	http://192.168.56.2/		
01:48:14...	HTTP	→ Request	GET	http://192.168.56.2/4dm1n/		
01:48:14...	HTTP	→ Request	POST	http://example.com/		
01:48:28...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv4		
01:48:28...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv6		
01:54:23...	HTTP	→ Request	GET	http://192.168.56.2/4dm1n/		
01:54:23...	HTTP	→ Request	POST	http://example.com/		

Request

Pretty Raw Hex

```
1 POST / HTTP/1.1
2 Host: example.com
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-type: application/x-www-form-urlencoded
8 Content-Length: 54
9 Origin: http://192.168.56.2
10 Connection: keep-alive
11 Referer: http://192.168.56.2/
```

Inspector

Request body parameters

Name	Value
flag 6 ->	0b318db8f0d5b38c6d38ede5e2af71c3

Request cookies

Event log (14) All issues Memory: 126.2MB Disabled

← → ↻ 192.168.56.2/4dm1n/? OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Burp Suite Community Edition v2025.3.4 - Temporary Project

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to http://example.com:80 [unknown host] Open browser ?

Time	Type	Direction	Method	URL	Status code	Length
00:06:18...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv6		
00:06:19...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv4		
00:06:27...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv4		
00:06:27...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv6		
00:16:49...	HTTP	→ Request	GET	http://192.168.56.2/		
01:48:14...	HTTP	→ Request	GET	http://192.168.56.2/4dm1n/		
01:48:14...	HTTP	→ Request	POST	http://example.com/		
01:48:28...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv4		
01:48:28...	HTTP	→ Request	GET	http://detectportal.firefox.com/success.txt?pv6		
01:54:23...	HTTP	→ Request	GET	http://192.168.56.2/4dm1n/		
01:54:23...	HTTP	→ Request	POST	http://example.com/		

Request

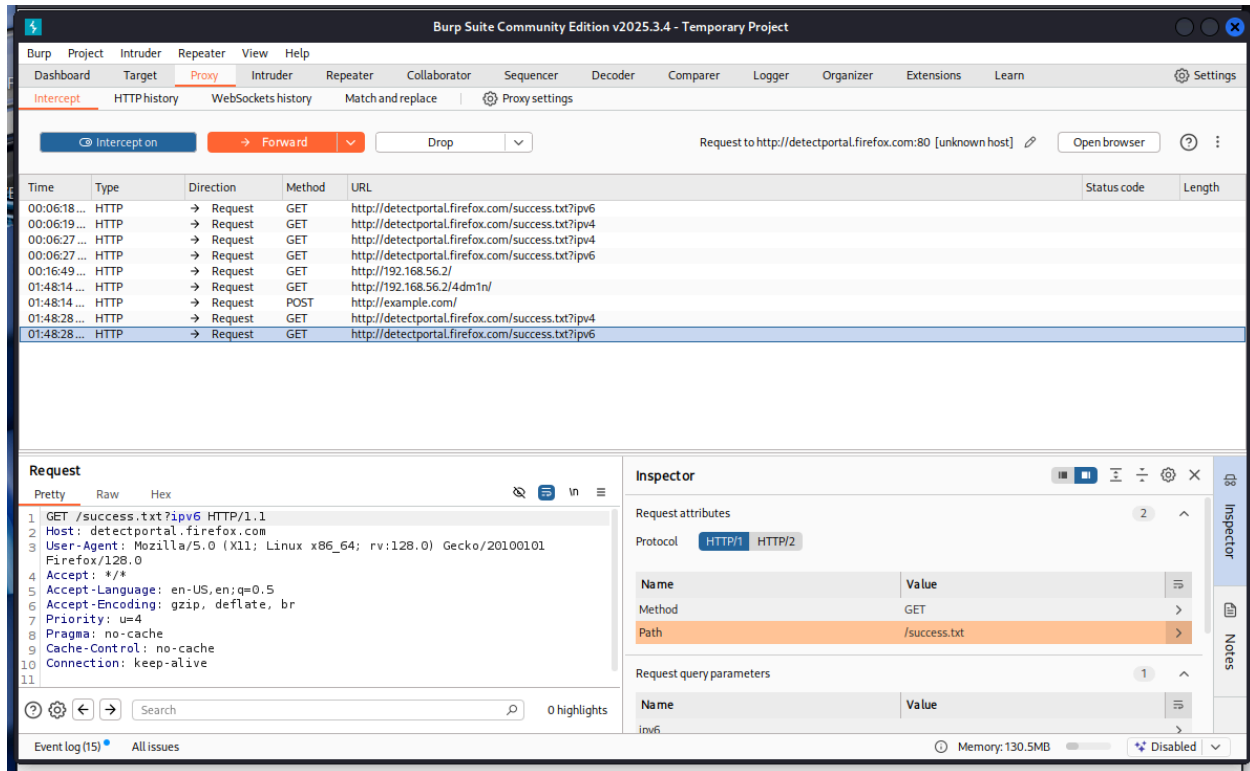
Pretty Raw Hex

```
6 Accept-Encoding: gzip, deflate, br
7 Content-type: application/x-www-form-urlencoded
8 Content-Length: 54
9 Origin: http://192.168.56.2
10 Connection: keep-alive
11 Referer: http://192.168.56.2/
12 ----
13 flag 6 -> 0b318db8f0d5b38c6d38ede5e2af71c3
14 ----
15 ----
```

Inspector

Host	example.com
User-Agent	Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/2...
Accept	*/*
Accept-Language	en-US,en;q=0.5
Accept-Encoding	gzip, deflate, br
Content-type	application/x-www-form-urlencoded
Content-Length	54
Origin	http://192.168.56.2
Connection	keep-alive
Referer	http://192.168.56.2/

Event log (14) All issues Memory: 126.2MB Disabled

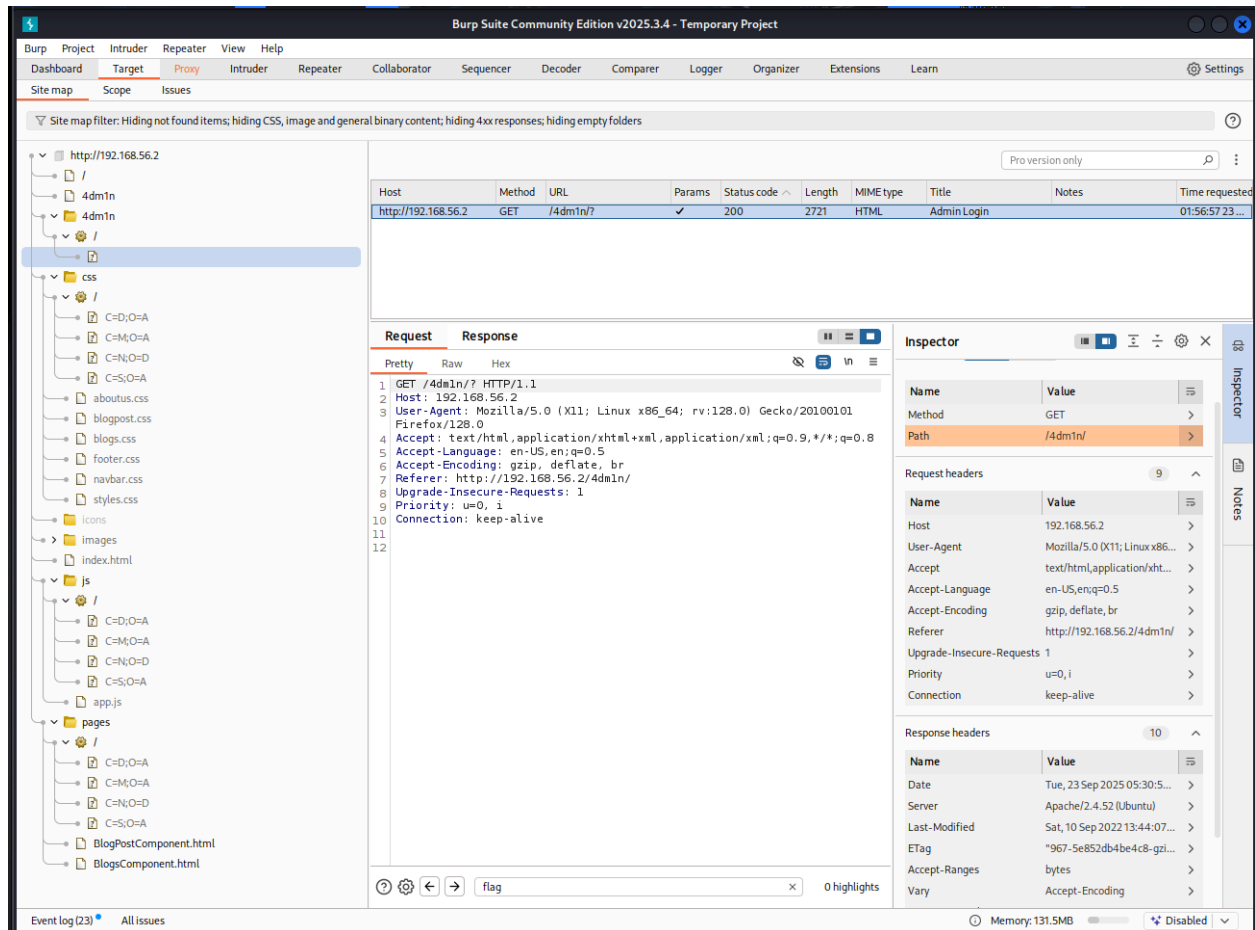


- Click on forward button
- Next the page admin page will open in the browser
- Once that opens enter the username password (guess it)
- Click on Login button on the admin web page
- Look at the Inspector section of the Burp suite
- We can find the flag 6 (possible sensitive information will be found here) as proven by the screenshot.
flag 6 -> 0b31bd8f0d5b38c6d38ede5e2af71c3

Monitor the Site Map in Burp

- Go to the Target tab in Burp Suite.
- Under Site map, expand the target host (e.g., http://192.168.56.2).
- As we manually browse, new URLs, parameters, and forms will appear automatically in the tree.

- Each page/component touch is captured for later review and



attack.

Use Burp's "Proxy history" for Deeper Review

- In Proxy > HTTP history, see every request/response captured by Burp, including details like parameters, cookies, headers, HTTP methods, and server responses.
- Send interesting requests to Burp's Repeater or Intruder for further testing.

The screenshot displays the Burp Suite interface. The top menu bar includes Burp, Project, Intruder, Repeater, View, and Help. The main toolbar shows various tools like Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Learn. The 'Proxy' tab is active, showing a list of intercepted requests. The 'Request' pane on the left shows the details of a GET request to /js/. The 'Response' pane in the middle shows the corresponding HTML response, which is an index page for /js/. The 'Inspector' pane on the right shows the request and response headers.

Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
GET	/css/			200	2155	HTML		Index of /css			192.168.56.2		01:44:05.23...	8080	2
GET	/favicon.ico			404	491	HTML	ico	404 Not Found			192.168.56.2		00:03:08.2...	8080	5
GET	/js/			200	1150	HTML		Index of /js			192.168.56.2		01:44:16.23...	8080	2
GET	/pages/			200	1405	HTML		Index of /pages			192.168.56.2		01:43:29.23...	8080	3
GET	/pages/BlogPostComponent.html			200	4686	HTML	html	California			192.168.56.2		01:47:09.23...	8080	2
GET	/pages/BlogsComponent.html			200	3771	HTML	html	Blog Posts			192.168.56.2		01:47:11.23...	8080	
GET	/success.txt?pv4		✓			text	txt				unknown host		00:06:19.23...	8080	
GET	/success.txt?pv4		✓			text	txt				unknown host		00:06:27.23...	8080	
GET	/success.txt?pv4		✓			text	txt				unknown host		01:48:28.23...	8080	
GET	/success.txt?pv6		✓			text	txt				unknown host		00:06:18.23...	8080	
GET	/success.txt?pv6		✓			text	txt				unknown host		00:06:27.23...	8080	
GET	/success.txt?pv6		✓			text	txt				unknown host		01:48:28.23...	8080	

Request

```

1 GET /js/ HTTP/1.1
2 Host: 192.168.56.2
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Upgrade-Insecure-Requests: 1
9 Priority: u=0, i
10
11

```

Response

```

1 HTTP/1.1 200 OK
2 Date: Tue, 23 Sep 2025 05:22:11 GMT
3 Server: Apache/2.4.52 (Ubuntu)
4 Vary: Accept-Encoding
5 Content-Length: 923
6 Keep-Alive: timeout=5, max=100
7 Connection: Keep-Alive
8 Content-Type: text/html; charset=UTF-8
9
10 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2 Final//EN">
11 <html>
12 <head>
13 <title>
14   Index of /js
15 </title>
16 </head>
17 <body>
18 <table>
19 <tr>
20 <th valign="top">
21 
22 </th>
23 <th>
24 <a href="?C=N;O=D">
25   Name
26 </a>
27 </th>
28 <th>
29 <a href="?C=M;O=A">

```

Inspector

Request headers

Name	Value
Host	192.168.56.2
User-Agent	Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/2...
Accept	text/html,application/xhtml+xml,application/xml...
Accept-Language	en-US,en;q=0.5
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Upgrade-Insecure-Requests	1
Priority	u=0, i

Response headers

Name	Value
Date	Tue, 23 Sep 2025 05:22:11 GMT
Server	Apache/2.4.52 (Ubuntu)
Vary	Accept-Encoding
Content-Length	923
Keep-Alive	timeout=5, max=100
Connection	Keep-Alive
Content-Type	text/html; charset=UTF-8

Notes

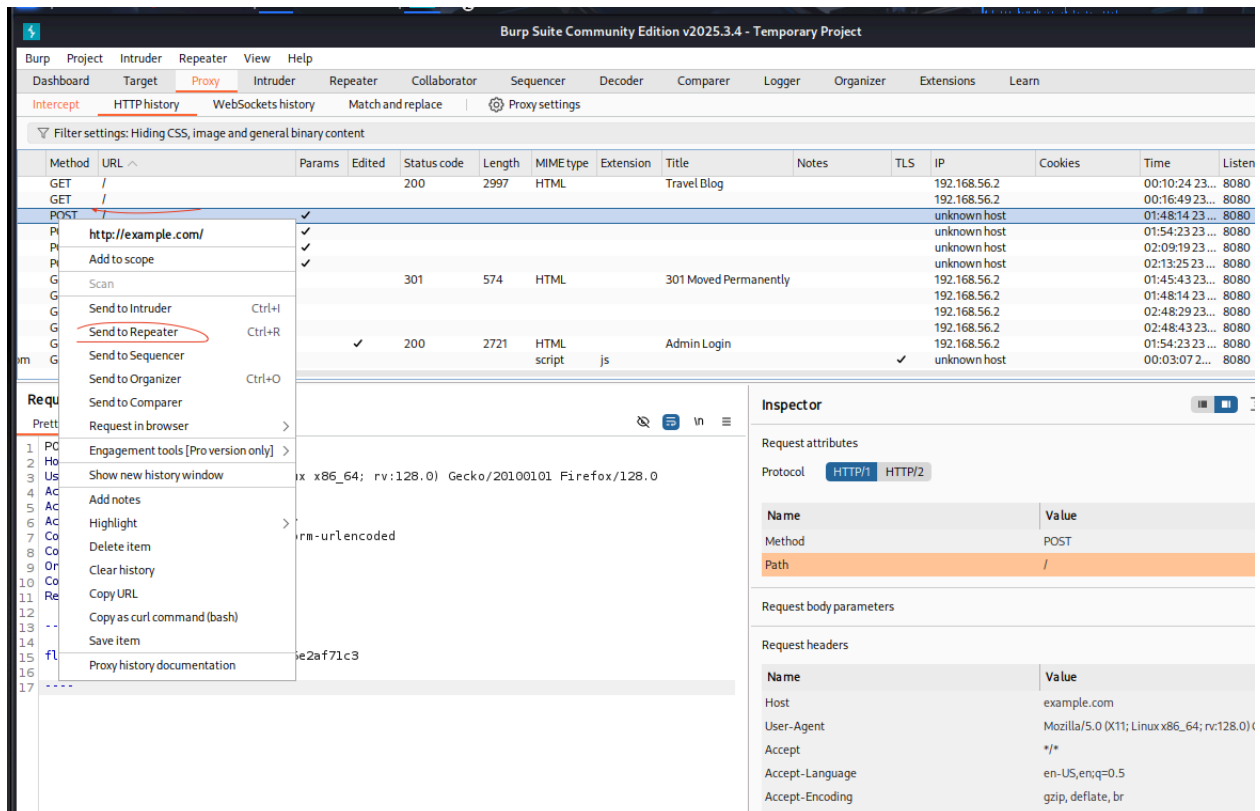
- Manual crawling is crucial for authentication-required or JavaScript-heavy apps where automated crawlers often miss things.
- Each dynamic element discovered becomes a potential attack vector for vulnerabilities (XSS, SQLi, CSRF, IDOR).

4adm1n

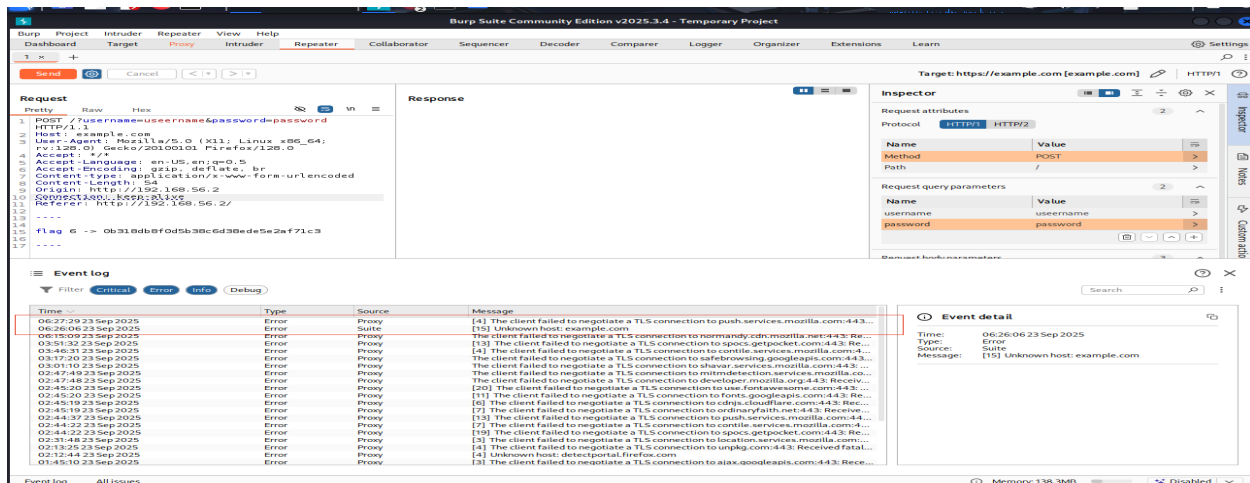
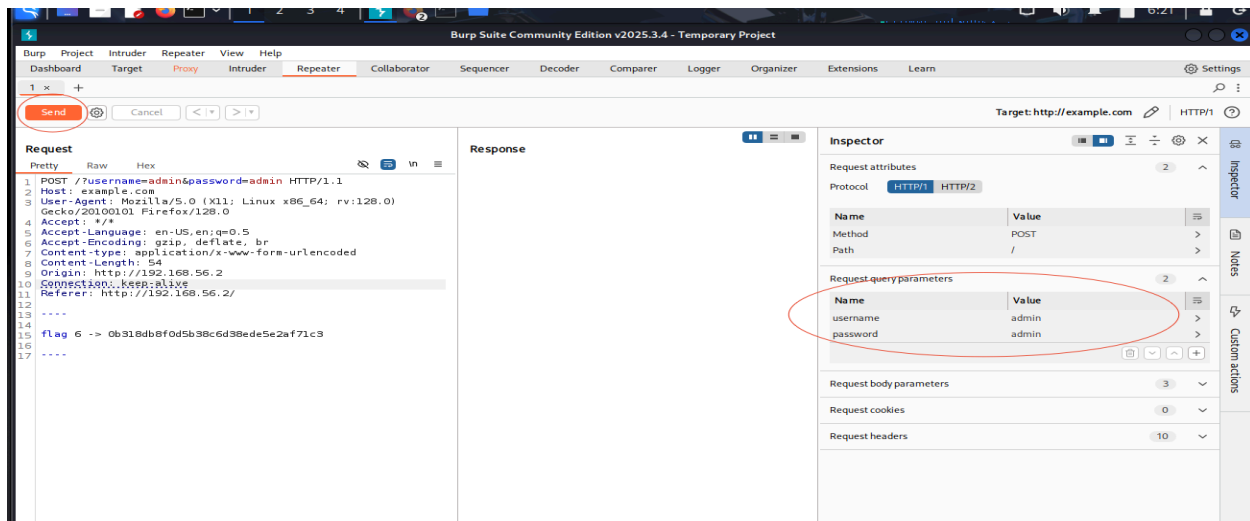
- Login form, POST captured
- test SQLi, weak/default creds, bypass
- Use Intruder/Repeater to fuzz parameters

How to test?

- Send the Captured Request to Repeater
- In Burp's Proxy > HTTP history, right-click the POST request (the login attempt).
- Select Send to Repeater.
- Switch to the Repeater tab.



- In the repeater → inspector window → request query parameter → click add → enter the username and password → click send.
- Check response. There is no response; rather when we open the event log we see the error that is generated. It shows an unknown host error.
- In Burp Suite, an "unknown host" error means that the target hostname cannot be resolved to an IP address, preventing the connection. So we have a dummy target. And so we cannot check sql injection or such attacks and we move to next page.



/pages/*.html

- Static blog pages with possible hidden info/comments
- Manual source/code review for flags, info

Burp Suite Community Edition v2025.3.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

Host Method URL Params Status code Length MIME type Title Notes Time requested

Host	Method	URL	Params	Status code	Length	MIME type	Title	Notes	Time requested
http://192.168.56.2	GET	/pages/BlogPostCompo...		200	4686	HTML	California		01:48:17.23...

Request Response

1 GET /pages/BlogPostComponent.html HTTP/1.1

2 Host: 192.168.56.2

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Connection: keep-alive

8 Referer: http://192.168.56.2/pages/

9 Upgrade-Insecure-Requests: 1

10 Priority: u=0, i

Inspector

Path /pages/BlogPostComponent.html

Request headers

Name	Value
Host	192.168.56.2
User-Agent	Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language	en-US,en;q=0.5
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Referer	http://192.168.56.2/pages/
Upgrade-Insecure-Requests	1
Priority	u=0, i

Response headers

Name	Value
Date	Tue, 23 Sep 2025 05:22:18 GMT
Server	Apache/2.4.52 (Ubuntu)
Last-Modified	Thu, 08 Sep 2022 18:23:14 GMT
ETag	"1114-5e82e85cf43c-gzip"
Accept-Ranges	bytes
Vary	Accept-Encoding
Content-Length	4372
Keep-Alive	timeout=5, max=97
Connection	Keep-Alive

Event log All Issues

Memory: 137.9MB Disabled

Burp Suite Community Edition v2025.3.4 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

Site map filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

Host Method URL Params Status code Length MIME type Title Notes Time requested

Host	Method	URL	Params	Status code	Length	MIME type	Title	Notes	Time requested
http://192.168.56.2	GET	/pages/BlogsCompone...		200	3771	HTML	Blog Posts		01:48:23.23...

Request Response

1 GET /pages/BlogsComponent.html HTTP/1.1

2 Host: 192.168.56.2

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

7 Connection: keep-alive

8 Referer: http://192.168.56.2/pages/

9 Upgrade-Insecure-Requests: 1

10 Priority: u=0, i

Inspector

Request headers

Name	Value
Host	192.168.56.2
User-Agent	Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language	en-US,en;q=0.5
Accept-Encoding	gzip, deflate, br
Connection	keep-alive
Referer	http://192.168.56.2/pages/
Upgrade-Insecure-Requests	1
Priority	u=0, i

Response headers

Name	Value
Date	Tue, 23 Sep 2025 05:22:24 GMT
Server	Apache/2.4.52 (Ubuntu)
Last-Modified	Thu, 08 Sep 2022 18:23:14 GMT
ETag	"d81-5e82e85cf43c-gzip"
Accept-Ranges	bytes
Vary	Accept-Encoding
Content-Length	3457
Keep-Alive	timeout=5, max=100
Connection	Keep-Alive
Content-Type	text/html

Event log All Issues

Memory: 138.8MB Disabled

Analysis of /pages/*.html Content

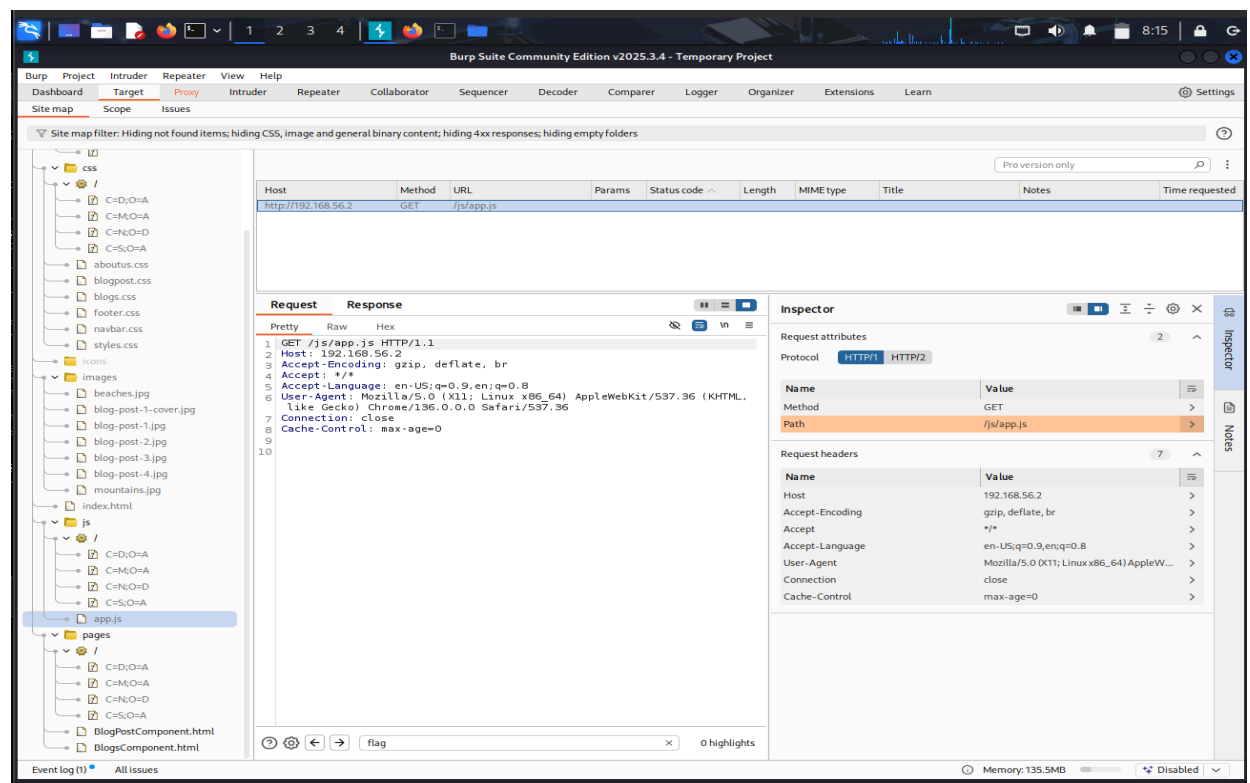
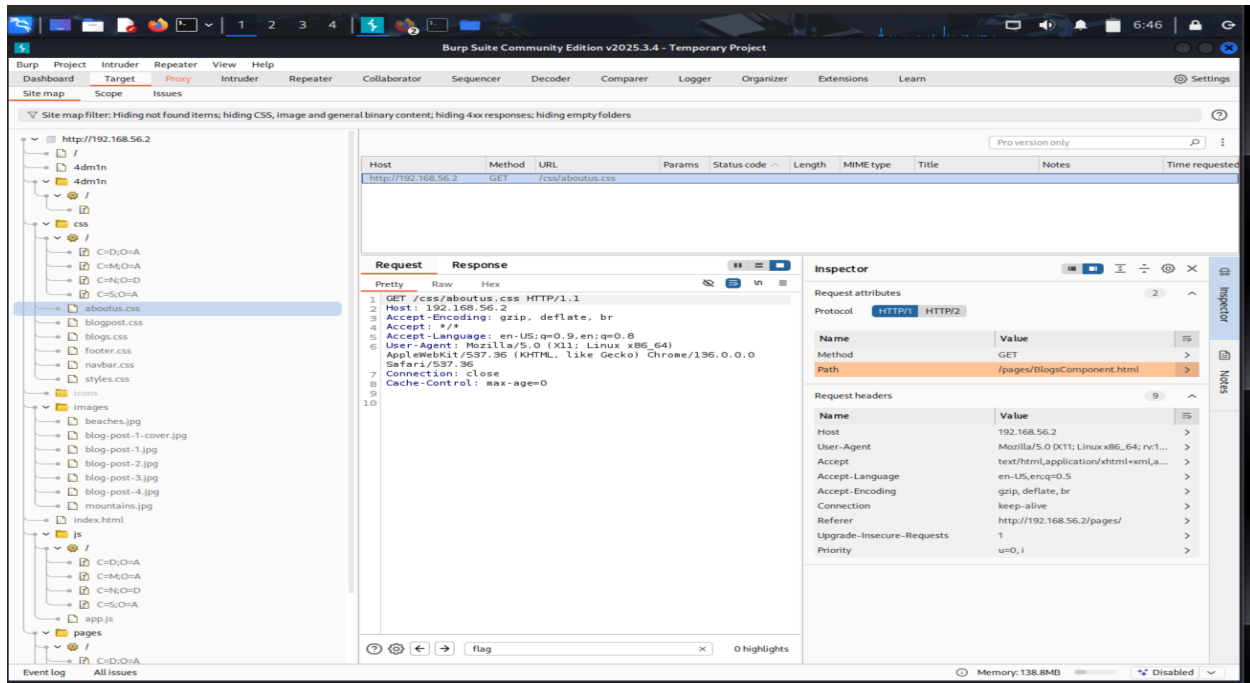
- The Burp requests show GETs to these HTML files, returning status 200 and presenting standard headers (e.g., Server: Apache/2.4.52 (Ubuntu)) and normal browser traffic.
- The responses are pure HTML; neither request or headers contain evidence of authentication challenge, sensitive cookies, nor backend API calls.

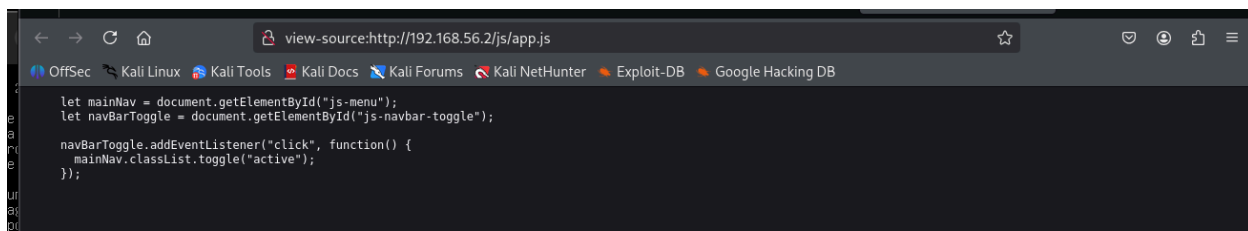
Hidden Information Assessment

- No Indication of Sensitive Data in Headers: The HTTP responses for these requests do not return cookies (e.g., no PHPSESSID, JWT, etc.) and have normal content-type and server data.
- File Names and Paths: The file names themselves (component-style .html files) do not reveal user credentials, tokens, or config info. These appear to be static blog component pages typical of a beginner-level vulnerable app.
- No Extra Parameters or Query Strings: Requests do not include suspicious or interesting query strings which sometimes indicate dynamic/back-end interaction that might leak info.

/css/, /js/

- Public static files. JS can leak endpoints, functionality
- Inspect for hidden functionality/endpoints





```
view-source:http://192.168.56.2/js/app.js
let mainNav = document.getElementById("js-menu");
let navBarToggle = document.getElementById("js-navbar-toggle");

navBarToggle.addEventListener("click", function() {
  mainNav.classList.toggle("active");
});
```

Analysis of /css/ and /js/app.js

- CSS Files (/css/aboutus.css)
 - Request: Standard GET request to aboutus.css
 - Response: Status 200, normal CSS content-type headers
 - Finding: No sensitive information visible in the request/response headers or traffic patterns
- JavaScript File (/js/app.js)
 - Request: Standard GET request to app.js
 - Response: Status 200, standard JavaScript content-type
 - Finding: No sensitive information visible in the HTTP traffic itself
- Analysis of Js file code - It contains Frontend code for
 - Navigation Toggle: Simple menu toggle functionality for a responsive navigation bar
 - DOM Manipulation: Basic JavaScript to show/hide a mobile menu when clicked
 - No Sensitive Data: No hardcoded credentials, API endpoints, or admin functionality

Burp suite: Vulnerabilities and Recommendations

Vulnerability	File/URL	Recommended Action
Reflected XSS in input	/4dm1n/ (HTML/PHP/JS)	• Sanitize user input before echoing. • Use HTML/JS escaping everywhere user data shows up in DOM. • Implement CSP where possible.
Form brute force possible	/4dm1n	• Implement account lockout, rate-limit login attempts, add CAPTCHA, log and alert after threshold failures.
Insecure HTTP only	All (/ , /4dm1n/ , etc.)	• Redirect all traffic to HTTPS with HSTS. Do not allow plain HTTP.
Lack of input validation	POST target	• Add server-side validation for all inputs, not just client-side. • Use allow-listing wherever possible
Information disclosure	Error messages, verbose API responses	• Customize error messages to be generic. • Do not expose backend system details to the user. • Log detailed errors server-side only.

OWASP ZAP

OWASP ZAP Introduction & Setup

What is OWASP ZAP?

- Free, open-source web application security scanner
- Similar to Burp Suite but completely free with full functionality
- Automated vulnerability scanning capabilities
- Great for finding OWASP Top 10 vulnerabilities

Why use ZAP after Burp?

- Compare results between two different tools
- ZAP might catch vulnerabilities Burp Community missed
- Provides automated scanning (unlike Burp Community's limited scanning)
- Good for comprehensive security assessment

ZAP Attack

Launch OWASP ZAP

```
zaproxy
```

```
(kalimainuser@kali)-[~]
$ zaproxy
Command 'zaproxy' not found, but can be installed with:
sudo apt install zaproxy
Do you want to install it? (N/y)y
sudo apt install zaproxy
[sudo] password for kalimainuser:
Installing:
  zaproxy

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1013
  Download size: 214 MB
  Space needed: 271 MB / 20.2 GB available

ZAP_2
Get:1 http://mirror.freedif.org/kali kali-rolling/main amd64 zaproxy all 2.16
.1-0kali1 [214 MB]
Fetched 214 MB in 12s (17.5 MB/s)
Selecting previously unselected package zaproxy.
(Reading database ... 411917 files and directories currently installed.)
Preparing to unpack .../zaproxy_2.16.1-0kali1_all.deb ...
Unpacking zaproxy (2.16.1-0kali1) ...
Setting up zaproxy (2.16.1-0kali1) ...
Processing triggers for kali-menu (2025.2.7) ...

(kalimainuser@kali)-[~]
$
```

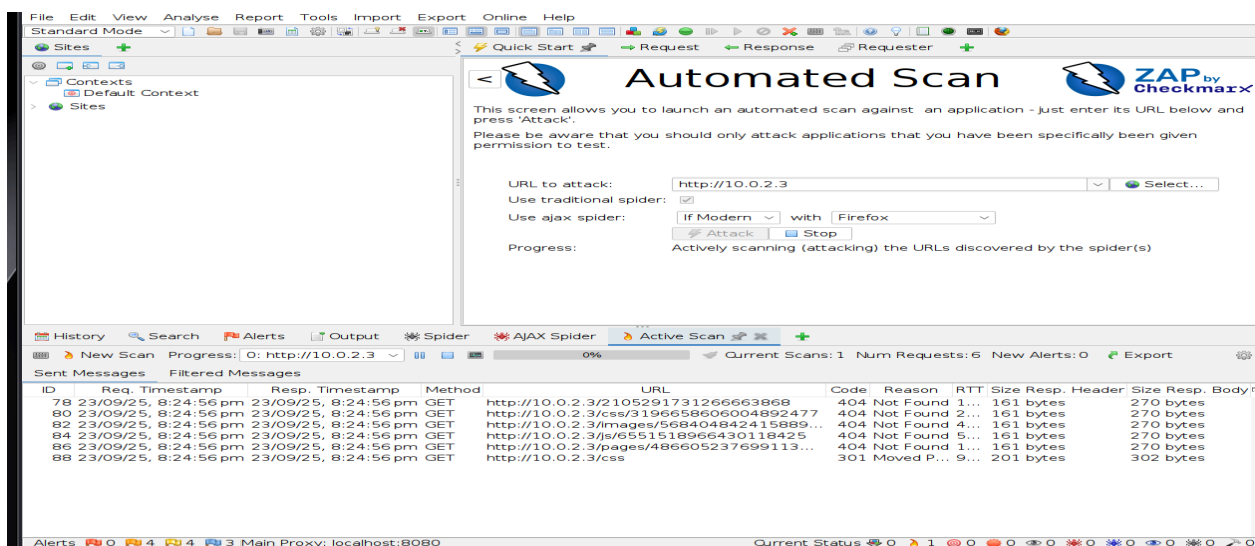
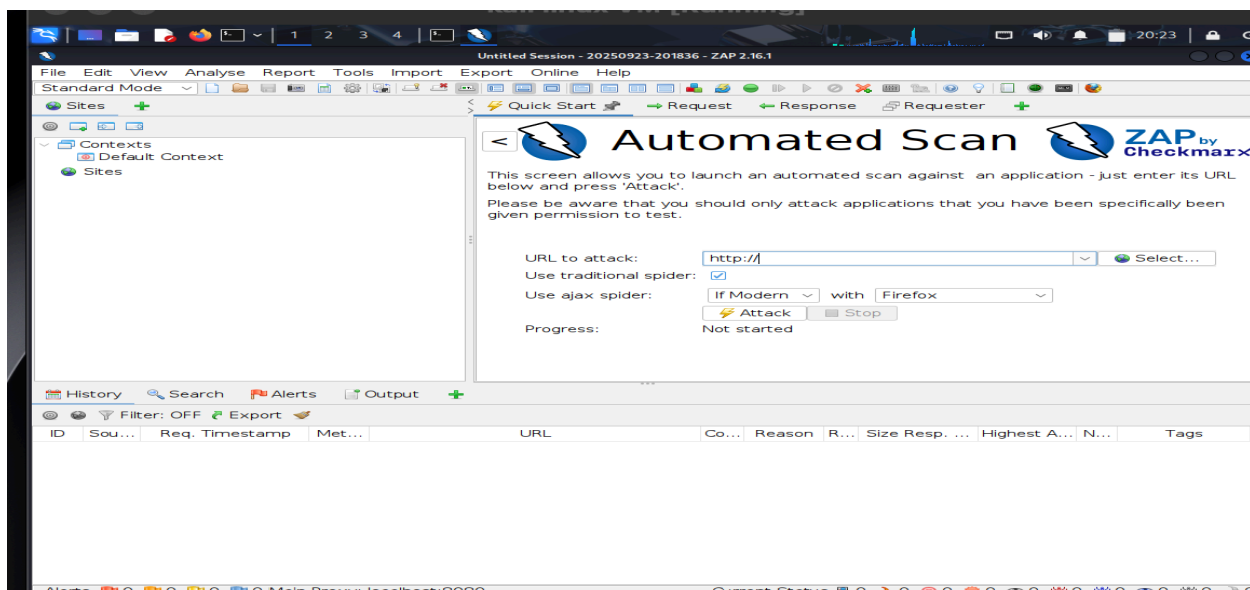
ZAP Starts:

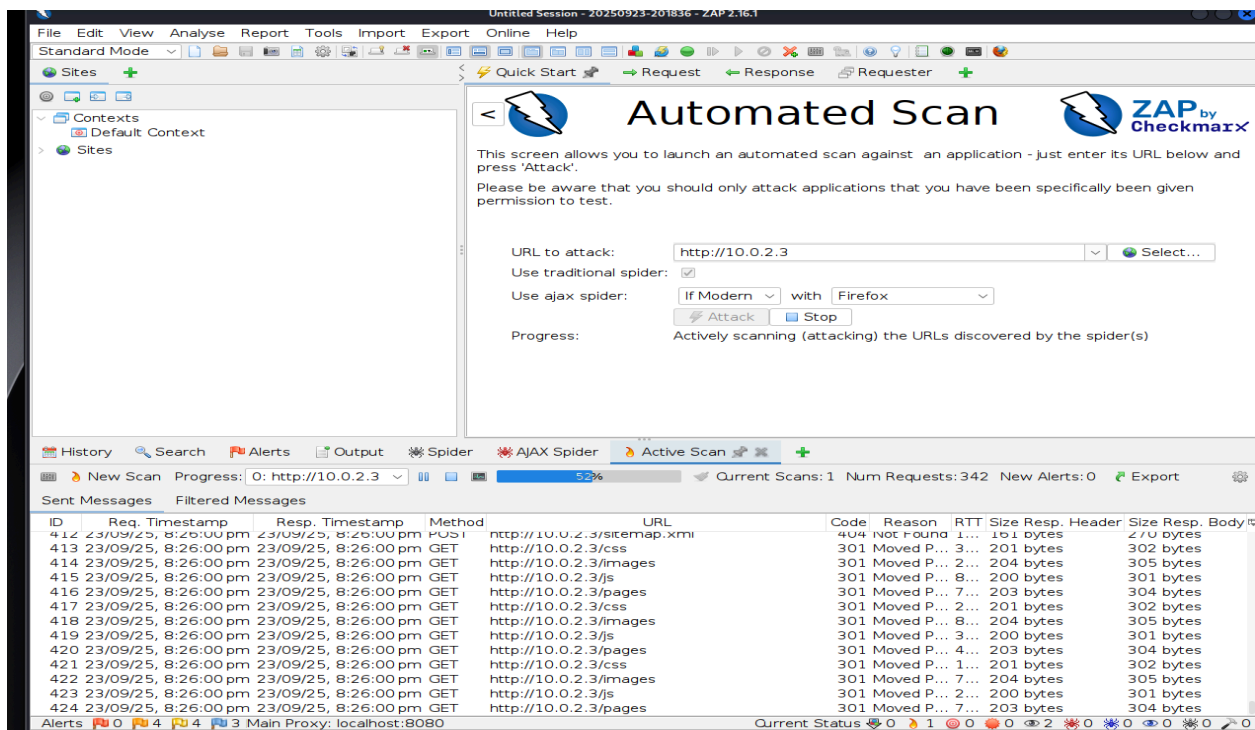
- Choose session type: Select "Persist Session" for saving results
- Configure proxy: ZAP will run on 127.0.0.1:8080 by default
- Set target URL: Enter the vulnerable VM's URL (e.g., `http://[NEW_IP]/`)

Note: Due to technical problems needed to install both VMs. So in upcoming screenshots we will use a new IP address for vulnerable VM.

ZAP Automated Scan

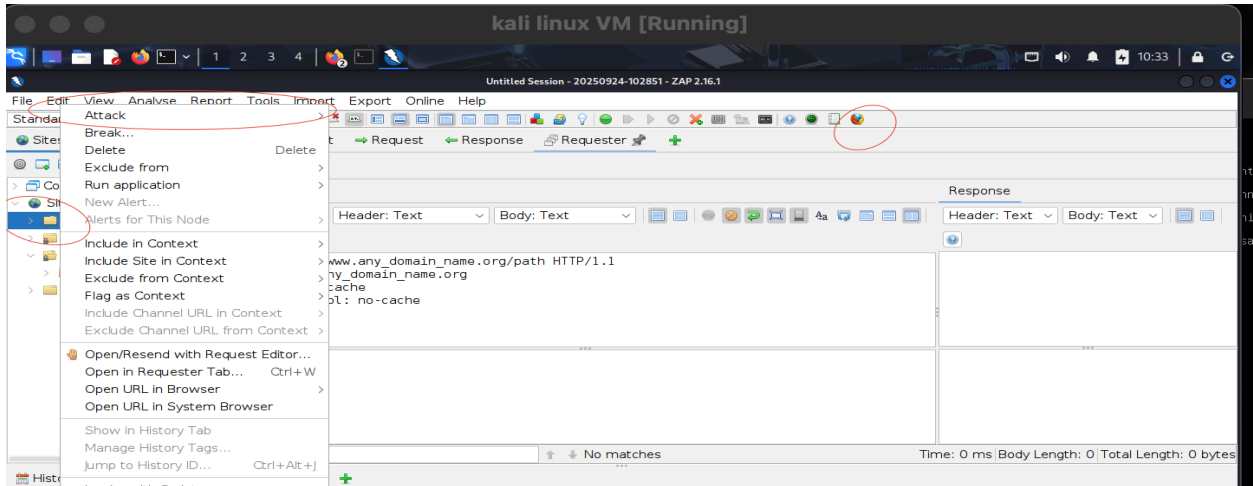
- Zap → Automated Scan

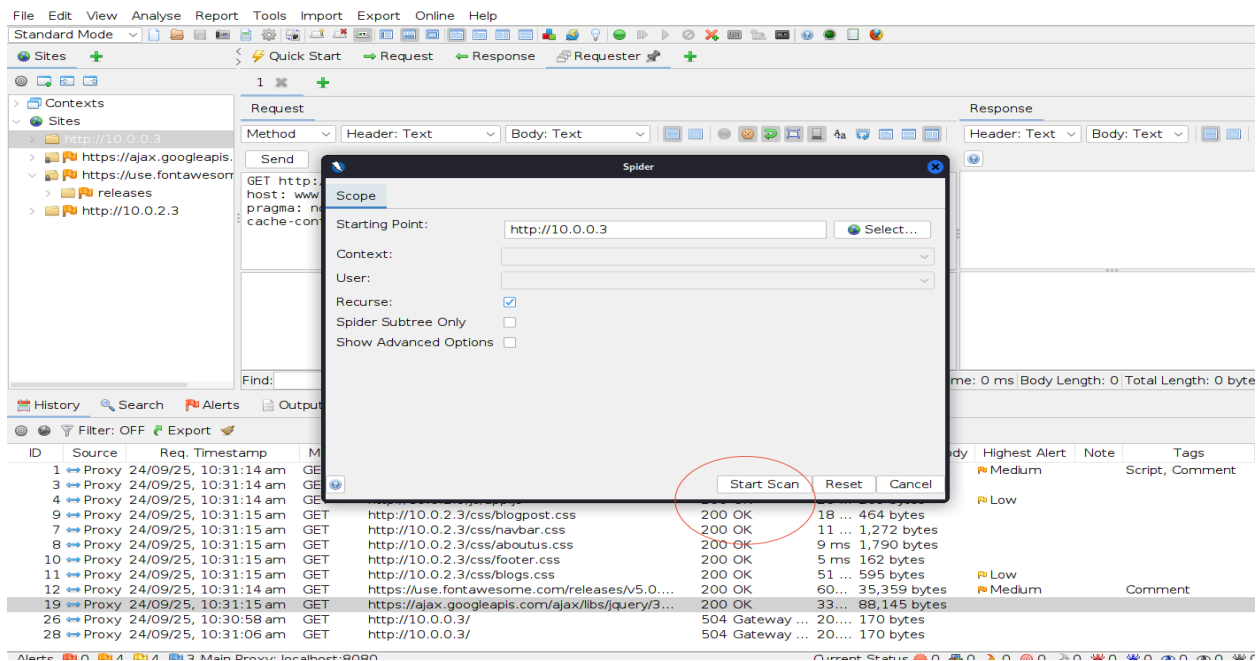
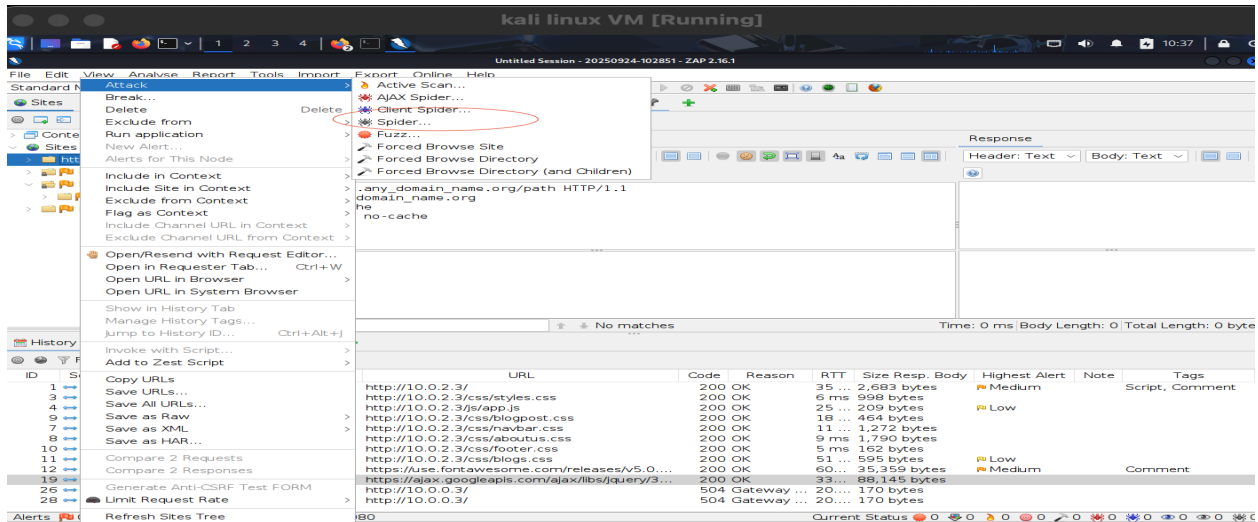




ZAP Spider Crawl

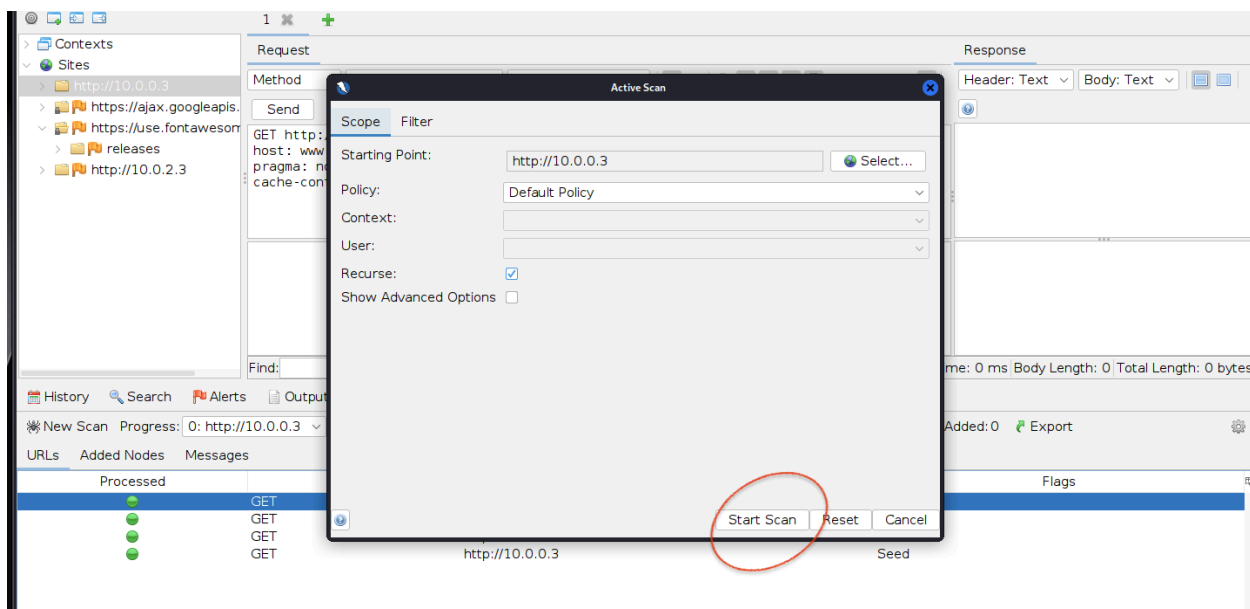
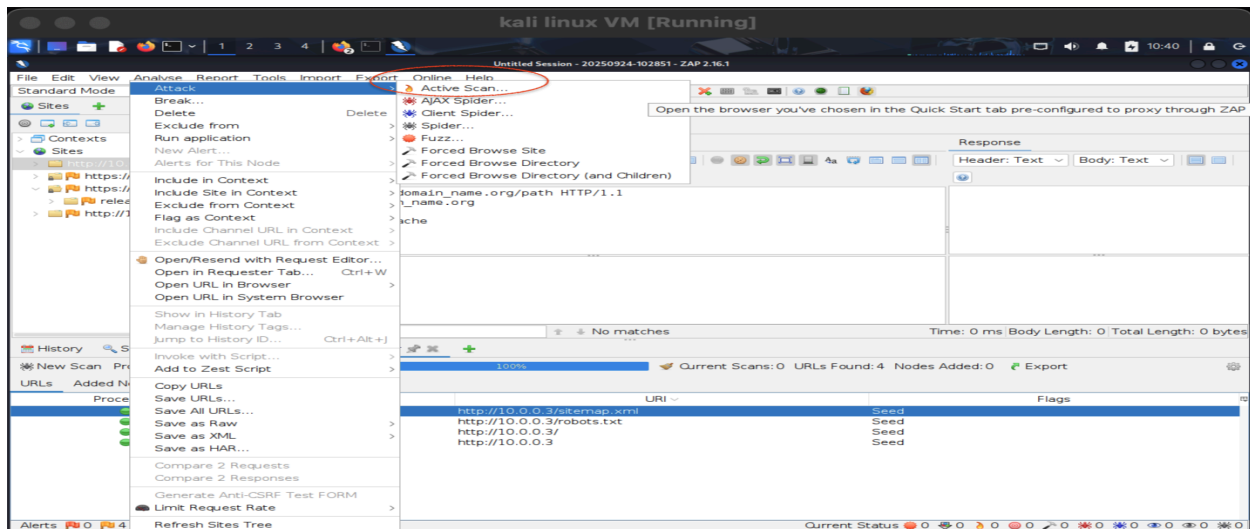
- Dashboard → firefox → http://10.0.2.3 (new IP of Vulnerable VM)
- Site Map → Right click on IP in the site map → Attack → Spider



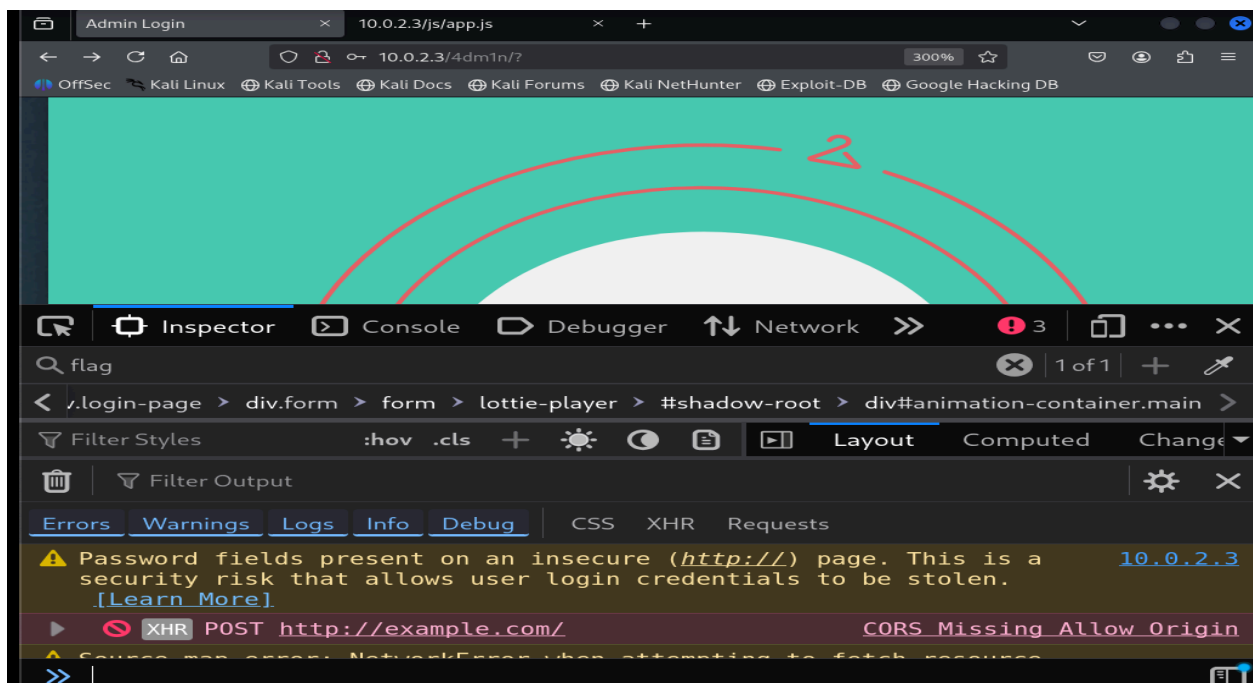
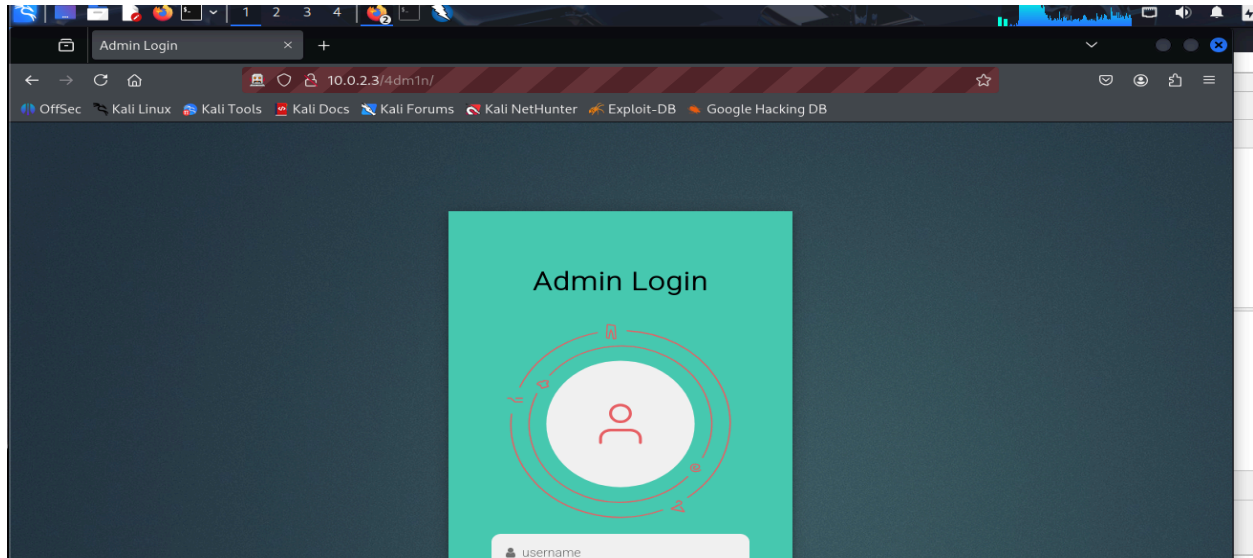


ZAP Active Scan

Site Map → Right Click on the IP Address of Site → Active Scan



Now open the admin page from the safari browser option in ZAP. Enter the admin page url as shown in the screenshot.





New Findings ZAP Discovered (Not Found in Burp):

Form with Password Field Flagged as "Medium" Risk

- Location: /4admin/ login form
- Risk Level: Medium
- Finding: "Form, Password..." vulnerability
- Why Burp Missed This: Burp's passive scanning didn't flag form security issues as prominently

Script/Comment Tags Flagged

- Multiple instances of "Script, Comment" findings
- Risk Level: Medium/Low
- Pages Affected: Multiple pages including blog components
- Issue: Potentially unsafe JavaScript comments or inline scripts

More Comprehensive Site Coverage

ZAP discovered and cataloged 26 URLs vs Burp's more limited manual crawling, including:

- /sitemap.xml (404 error)
- More blog component pages
- Better CSS/JS file enumeration

Error Page Analysis

- 404 errors properly flagged and analyzed
- Gateway timeout errors documented (504 errors)
- More systematic error handling analysis

Authentication Context Analysis

- ZAP shows authentication-related findings in the admin form
- Better analysis of login mechanisms
- Flag parameter detected in POST request (flag 6 -> 0b318db8f0d5b38c6d38ede5e2af71c3)

What This Reveals:

ZAP's Automated Advantages:

- Systematic Crawling: Found more pages automatically
- Form Analysis: Better at identifying form security issues and also great at application level security suggestions
- Comprehensive Scanning: More thorough passive + active scans
- Error Handling: Better documentation of error responses

Burp's Manual Advantages (from earlier):

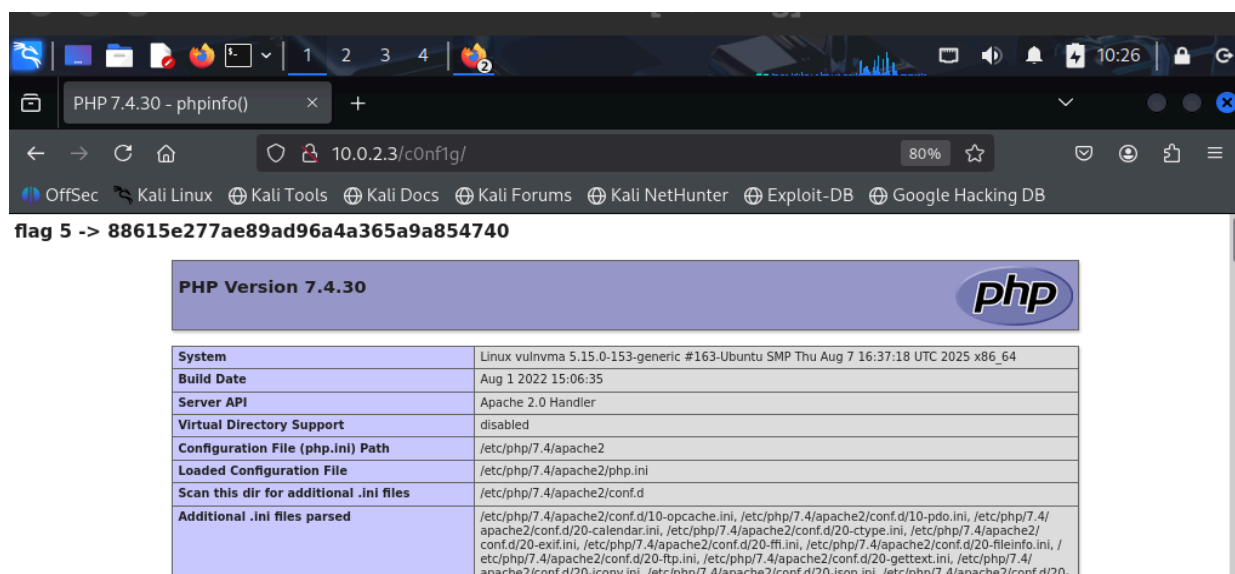
- Request Manipulation: Better for manual testing
- Detailed Inspection: More granular request/response analysis
- Custom Payloads: Manual injection testing capabilities

Most importantly, this taught us that we need to use a combination of tools in cyber security and must not rely on a single tool.

Flag 5

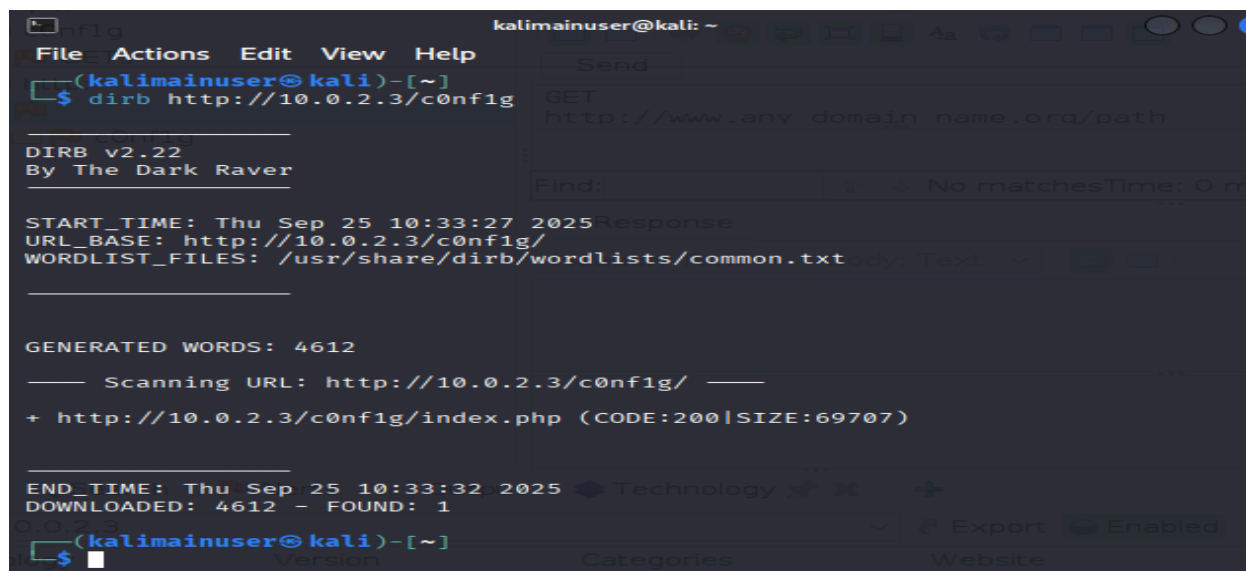
Since we found flags 1,2,3,4,6 - we needed to find Flag 5. It was later found in the c0nf1g page.

flag 5 -> 88615e277ae89ad96a4a365a9a854740



PHP Version 7.4.30	
System	Linux vulnvm 5.15.0-153-generic #163-Ubuntu SMP Thu Aug 7 16:37:18 UTC 2025 x86_64
Build Date	Aug 1 2022 15:06:35
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.4/apache2
Loaded Configuration File	/etc/php/7.4/apache2/php.ini
Scan this dir for additional .ini files	/etc/php/7.4/apache2/conf.d
Additional .ini files parsed	/etc/php/7.4/apache2/conf.d/10-opcache.ini, /etc/php/7.4/apache2/conf.d/10-pdo.ini, /etc/php/7.4/apache2/conf.d/20-calendar.ini, /etc/php/7.4/apache2/conf.d/20-ctype.ini, /etc/php/7.4/apache2/conf.d/20-exif.ini, /etc/php/7.4/apache2/conf.d/20-fileinfo.ini, /etc/php/7.4/apache2/conf.d/20-ftp.ini, /etc/php/7.4/apache2/conf.d/20-gettext.ini, /etc/php/7.4/apache2/conf.d/20-iconv.ini, /etc/php/7.4/apache2/conf.d/20-json.ini, /etc/php/7.4/apache2/conf.d/20-

Lets run dirb against this url.



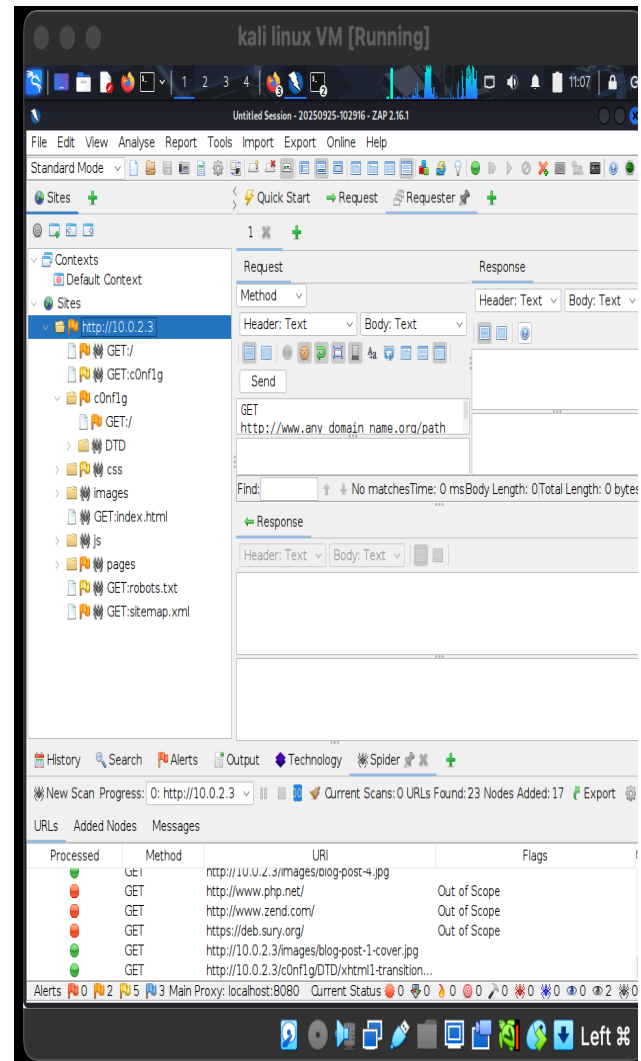
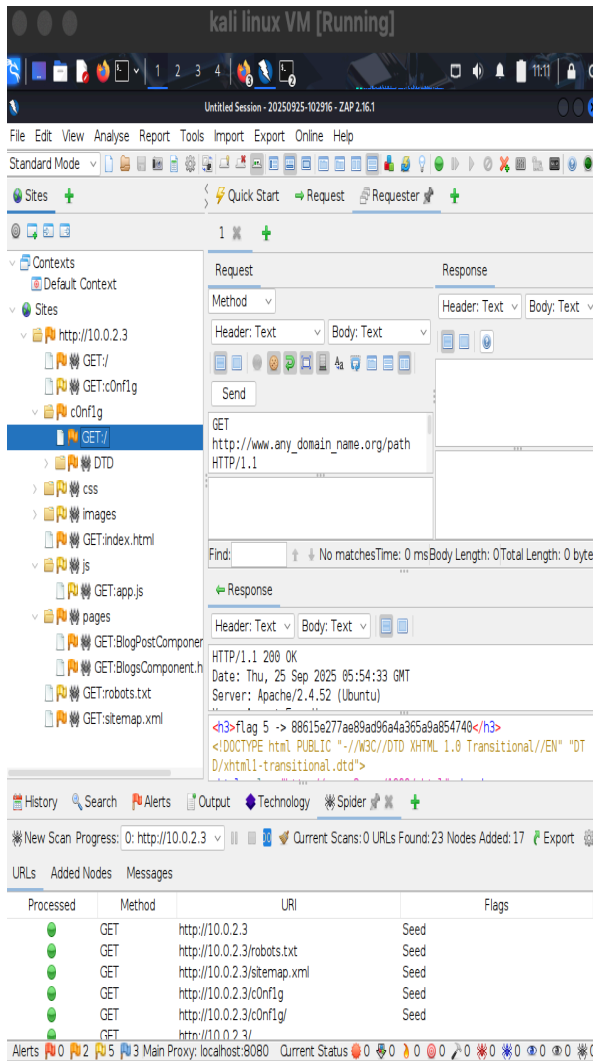
```
kalimainuser@kali: ~  
File Actions Edit View Help  
(kalimainuser@kali)-[~]  
$ dirb http://10.0.2.3/c0nf1g  
  
DIRB v2.22  
By The Dark Raver  
  
START_TIME: Thu Sep 25 10:33:27 2025  
URL_BASE: http://10.0.2.3/c0nf1g/  
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt  
  
GENERATED WORDS: 4612  
  
Scanning URL: http://10.0.2.3/c0nf1g/  
  
+ http://10.0.2.3/c0nf1g/index.php (CODE:200|SIZE:69707)  
  
END_TIME: Thu Sep 25 10:33:32 2025  
DOWNLOADED: 4612 - FOUND: 1  
  
(kalimainuser@kali)-[~]  
$
```

We found paths:

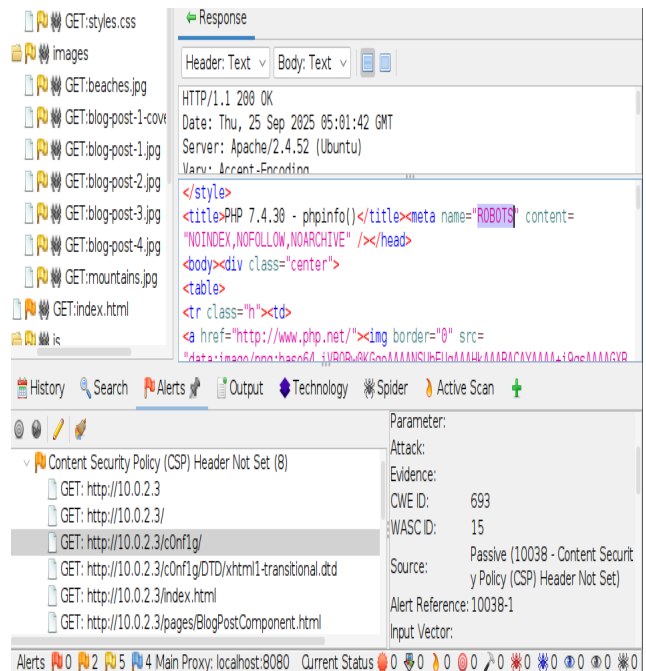
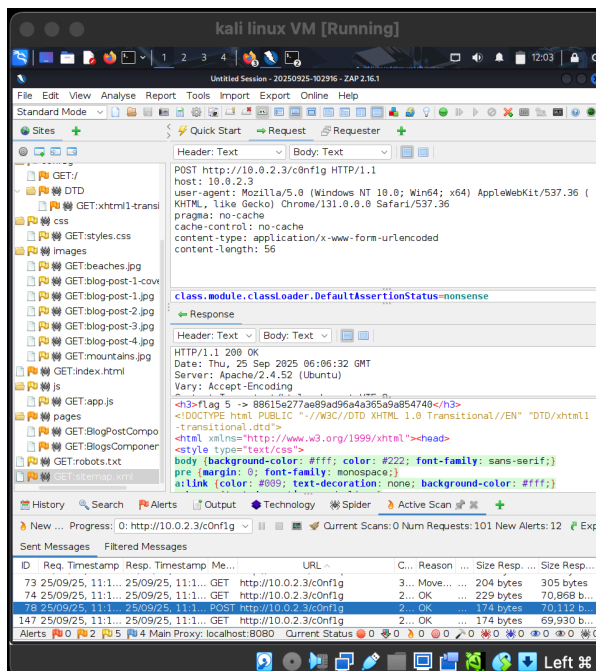
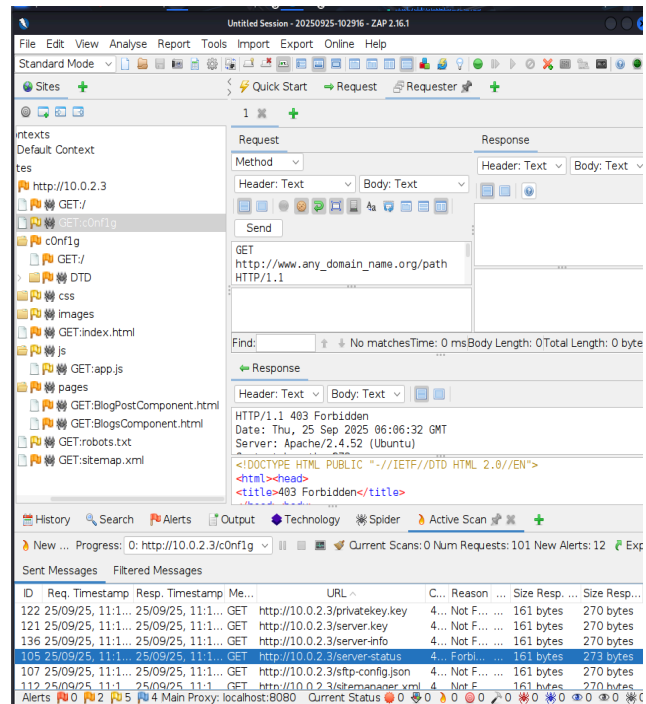
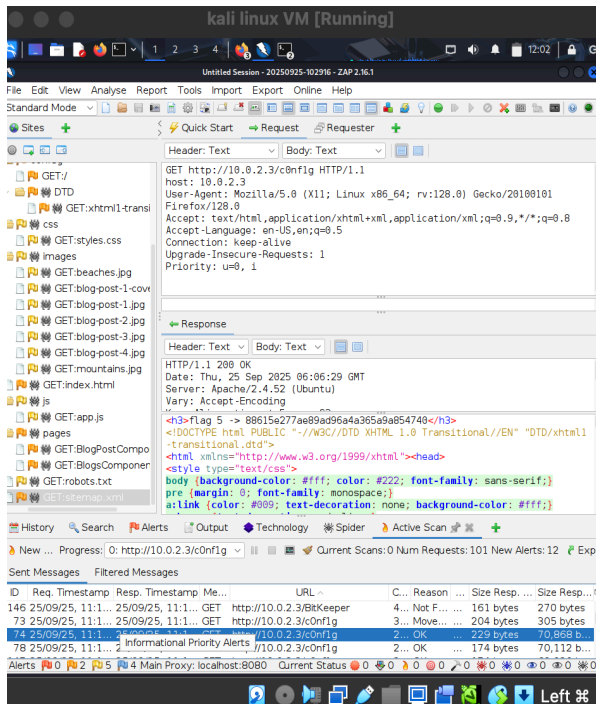
/c0ng1g

/c0nf1g/index.php

Next we use ZAP to perform spider scan this:



We also do active scan on this path:



Since there is nothing new here, we can move on with application of security fixes.

XSS Injection Testing on /4dm1n Admin Login

Testing Approach

To assess the security of the admin login page, several common Cross-Site Scripting (XSS) payloads were injected into the input fields—primarily the username box. The payload `<script>alert(1)</script>` was used as a representative test case. Submissions were performed using the browser interface as well as by capturing and modifying requests via developer tools and interception proxies.

Observation & Results

No XSS alert popups or script execution was observed in the browser after form submission, either in the login interface or in any error messages.

The server response remained unchanged regardless of input and did not reflect user-supplied data without sanitization.

The developer/network tool and ZAP proxy captures show "Access Denied" (403/401 errors) or generic login failures, with no evidence that XSS input affected page output or application flow.

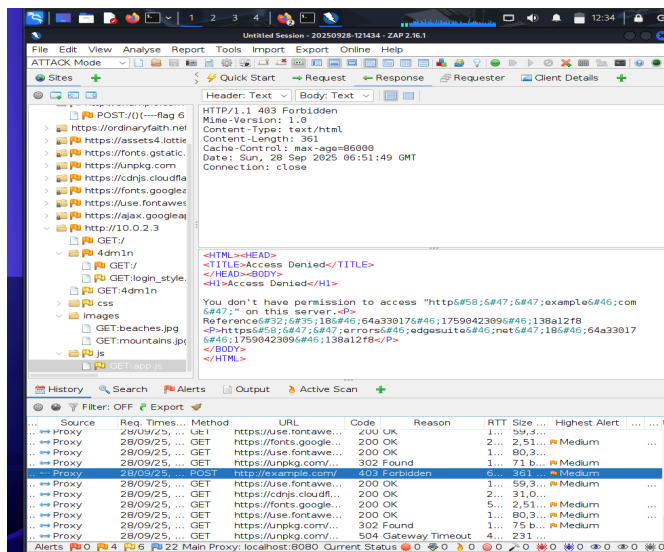
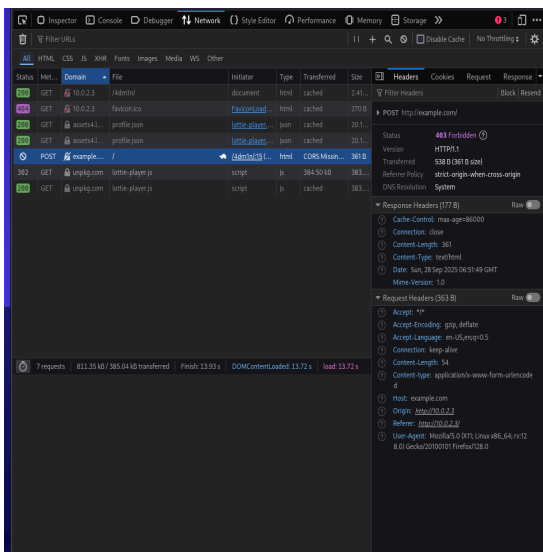
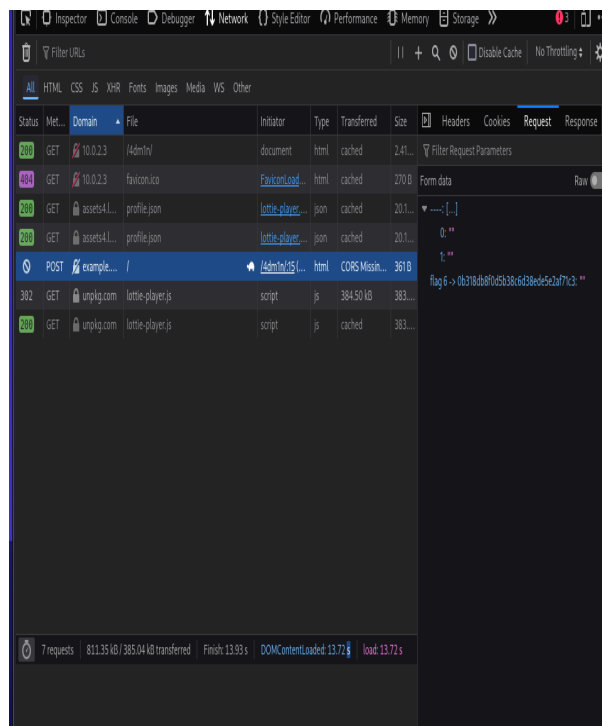
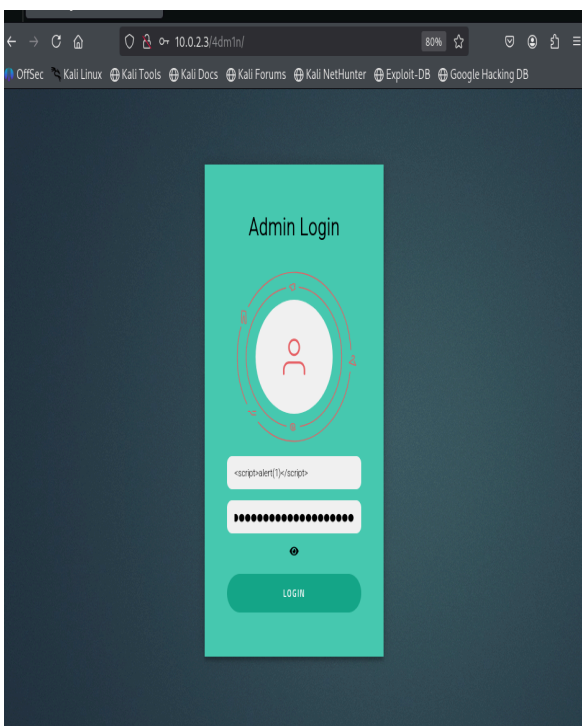
Conclusion

Based on this series of tests using script injection payloads, no XSS vulnerabilities were discovered in the login submission of the /4dm1n page. All injected scripts were either properly sanitized or not reflected in the output, and no code execution occurred on the client side. These findings indicate either proper input handling for XSS at

this entry point or the login request is simulated/fake and not processed at all.

Evidence Screenshots

Screenshots of the attempted attacks and system/network responses are attached here as documentation for the assessment.



SQL Injection Testing on /4dm1n Page

Testing Approach

Form credentials and payloads are submitted via POST requests to the login endpoint (/4dm1n) as seen in the captured network traffic.

The request includes a login attempt, with crafted input designed to test for SQL injection, such as special characters, quotes, or typical bypass strings.

The application responds with an HTTP 403 Forbidden status, blocking further interaction and stating "Access Denied," with no indication of backend SQL errors, leaks, or abnormal behavior.

The proxy logs confirm that requests are routed to the admin page and that all attempts are properly redirected or denied with status codes (e.g., 301 Moved Permanently, 403 Forbidden).

Observations

No SQL error messages or flawed behavior were displayed in the browser or network logs. This suggests that input is either sanitized before reaching the database, or the backend is simulated/fake and not processing queries at all.

The POST payload parameters (visible in the dev tools network view) do not appear to directly cause SQL syntax errors or unusual authentication bypasses.

The presence of strict server controls and redirect rules (status codes 403, 301) indicates defensive backend handling.

The lack of database-specific error messages is a positive sign, demonstrating that the backend either uses error handling best practices or is a simulation environment not fed by a real database.

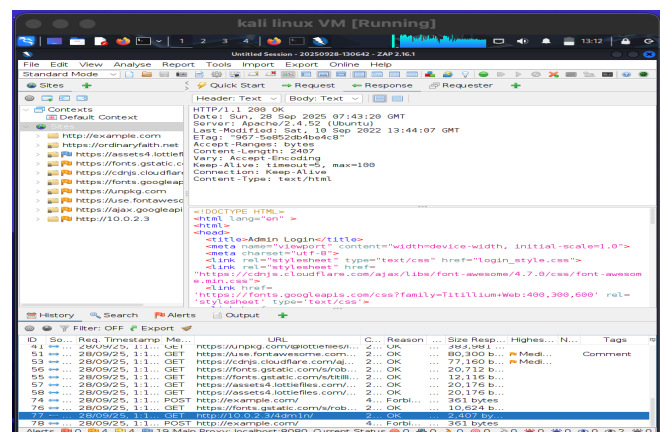
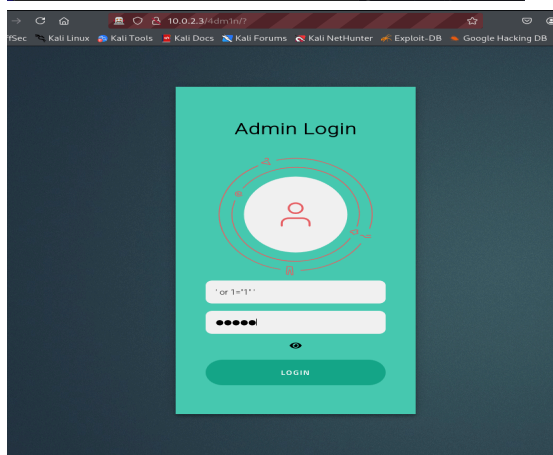
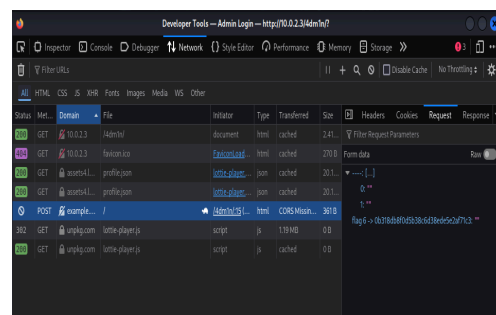
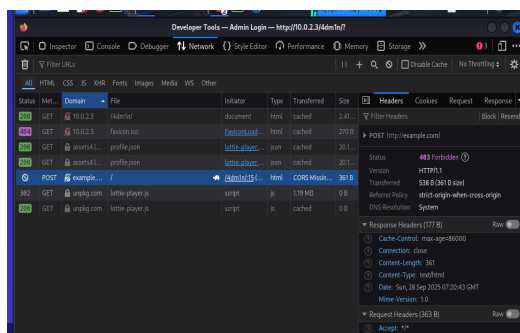
Conclusion

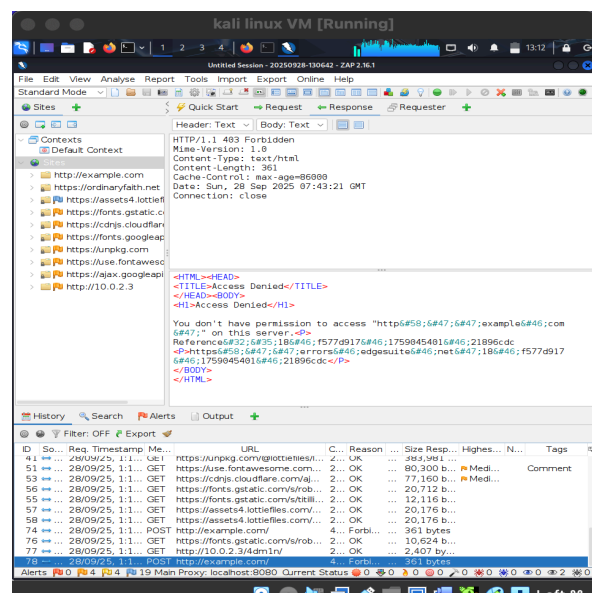
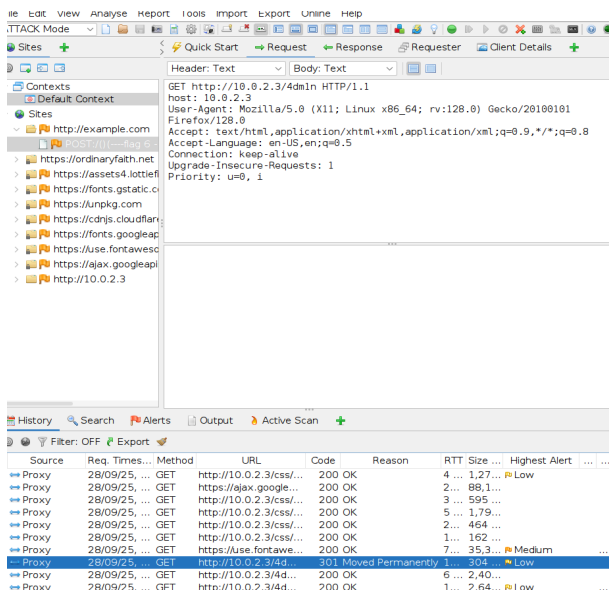
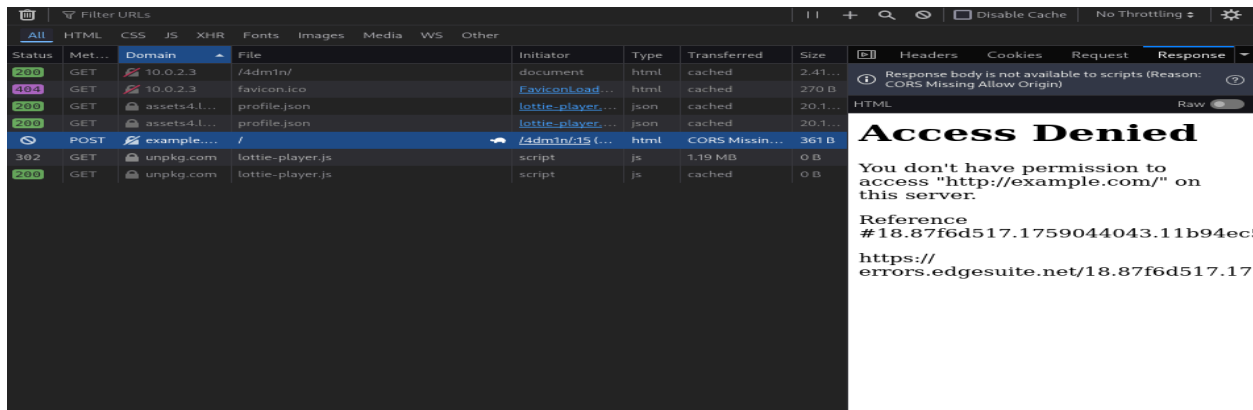
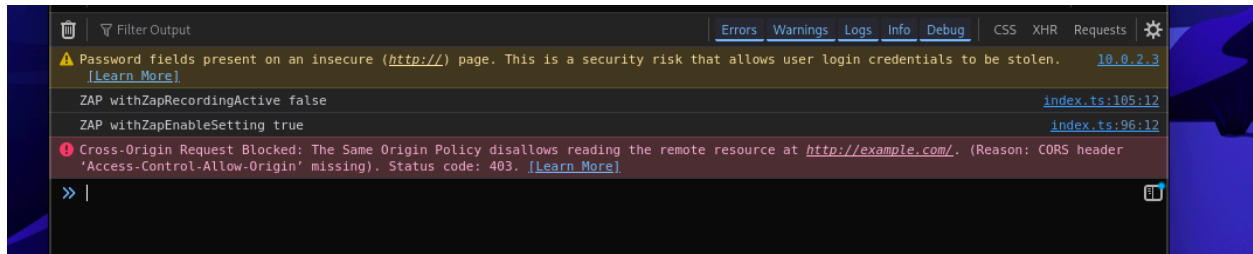
Despite submitting test payloads for SQL injection, the page did not reflect or leak sensitive database errors, nor did it allow bypass authentication via crafted input.

This simulated setup is either immune to SQL injection by virtue of non-existent backend logic or appropriate input filtering and error handling.

Evidence Screenshots

Screenshots of the attempted attacks and system/network responses are attached here as documentation for the assessment.





Next Steps in Security Assessment

Verification and Validation

- Retest the application after patches / fixes are applied by the development team.
- Use the same tools (Burp, ZAP, DIRB, manual testing) to confirm vulnerabilities have been fixed.
- Ensure no new issues have been introduced during remediation.

Knowledge Sharing and Improvement

- Discuss with developers and stakeholders about the lessons learned.
- Recommend improvements for secure development, regular vulnerability assessments, and awareness training.

Ongoing Monitoring and Hardening

- Propose to establish continuous monitoring for new vulnerabilities.
- Suggest periodic penetration testing and code reviews as part of the security lifecycle.
- These steps ensure our work directly leads to a more secure application and helps build a process for future security.

OWASP Vulnerabilities and Recommendations

OWASP Vulnerability	Vulnerability and Location	Recommended Action
Cryptographic Failures	- Form with password are not HTTPS /4dm1n/ HTML file, login form	- Implement HTTPS with a valid certificate. - Enforce HTTPS redirects. - Never accept credentials over plain HTTP.
Vulnerable and Outdated Components And Security Logging and Monitoring Failures	- Script/Comment Disclosure - Multiple HTML files (results pages, blog, etc.)	- Remove sensitive comments or debug info from JS/HTML. - Minimize/obfuscate public scripts.
Security Logging and Monitoring Failures	- Sensitive Info in Error Messages - Error/404 pages, /sitemap.xml	- Remove stack traces or verbose error output. - Show generic errors to users. - Log detailed errors server-side only.
Security Logging and Monitoring Failures	- Accessible JavaScript & CSS /js/app.js, /css/styles.css and others	- Review for sensitive info or comments. - Minify files for production. - Remove unused or debug code.
Security Misconfiguration	- Directory Listing /images/, /pages/	- Disable directory listing in web server. - Protect directories with .htaccess or equivalent.
Broken Access Control	Potential Admin Panel Exposure	- Rename admin URL. - Require login/multifactor.

	• /4dm1n/	• Restrict by IP, if possible. • Monitor failed logins.
Server-Side Request Forgery (SSRF) And Unvalidated Redirects and Forwards	• Unvalidated Redirects • "301 Moved Permanently"	• Inform users when they are about to leave the site with a redirect confirmation page. • Validate and sanitize all redirect parameters
Cryptographic Failures	• Password in POST over HTTP • /4dm1n/ login POST	• Never transmit credentials over plain HTTP. • Always use HTTPS. • Apply HTTP Strict Transport Security (HSTS).

References

References for Documentation and Downloads

OWASP ZAP Proxy

<https://www.zaproxy.org/>

Burp Suite Community Edition

<https://portswigger.net/burp>

DIRB Web Content Scanner

<https://tools.kali.org/web-applications/dirb>

Kali Linux Distribution

<https://www.kali.org/documentation/>

References for Security Vulnerability and Recommendations

OWASP Application Security Verification Standard

<https://owasp.org/www-project-application-security-verification-standard/>

Sensitive info disclosure (exposed PHP info/flag)

<https://www.php.net/manual/en/function.phpinfo.php#87803>

Missing Content Security Policy (CSP) header

<https://owasp.org/www-project-secure-headers/>

Directory listing enabled (info disclosure risk)

https://httpd.apache.org/docs/current/misc/security_tips.html#directory

OWASP: Cross Site Scripting (XSS)

<https://owasp.org/www-community/attacks/xss/>

OWASP: SQL Injection

https://owasp.org/www-community/attacks/SQL_Injection